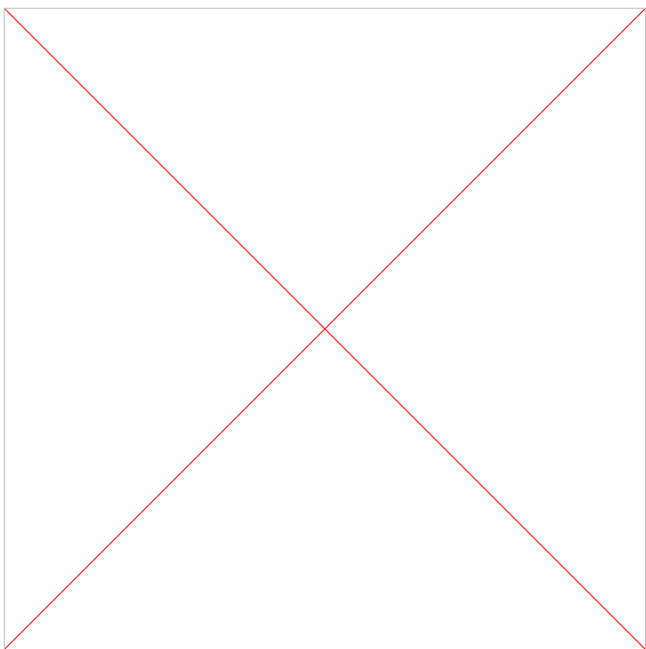


Isaac 體驗

Codpy Section 7

讓機器人在模擬世界學會走路

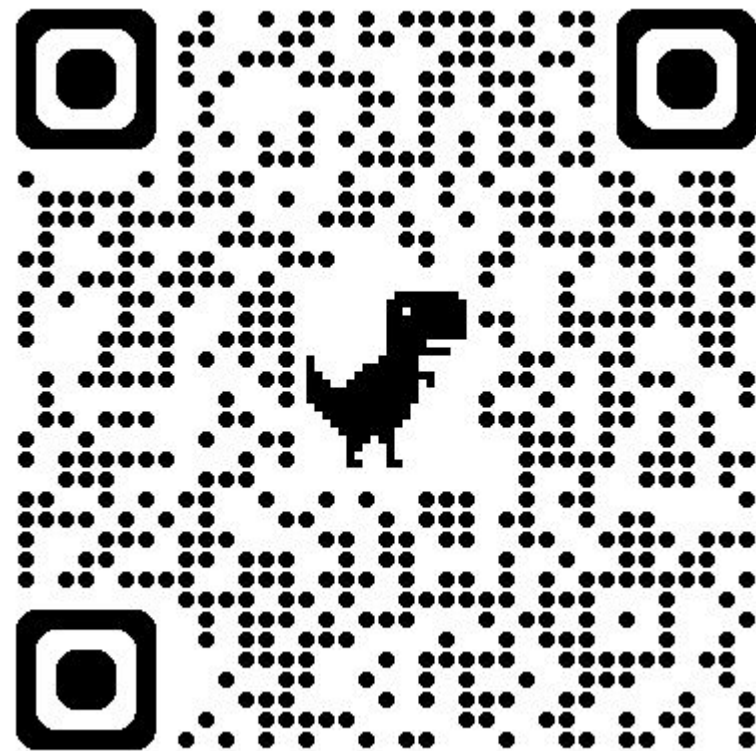


講者：王羿程

2026 / 09 / 27 · 成大旺宏館

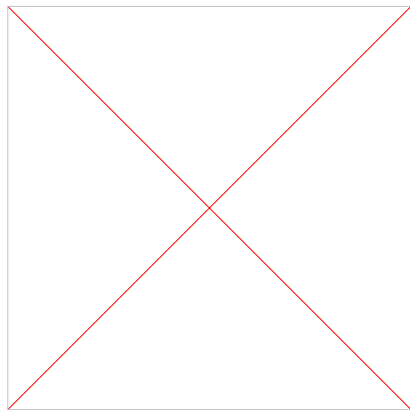
今日流程

09:00	認識 Isaac	
10:00	安裝 Isaac	實作
11:00	認識與匯入 URDF	實作
12:00	午餐	
13:00	強化學習	
14:00	訓練模型	實作



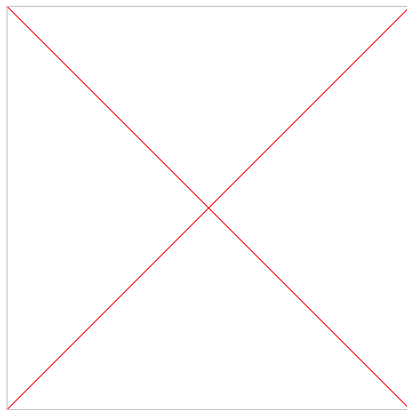
github

今天的目標



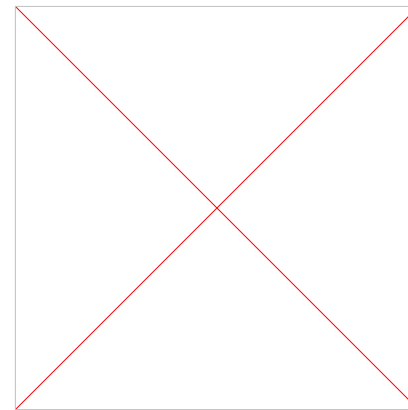
裝好 Isaac

Isaac Sim 6.0.1
Python 3.12



匯入機器人

URDF → USD
讓它站起來



訓練它走路

強化學習 PPO
看曲線、改獎勵

調查：

你的電腦有 NVIDIA 顯卡嗎？

(舉手)

還沒安裝？現在先開始下載

Windows · PowerShell

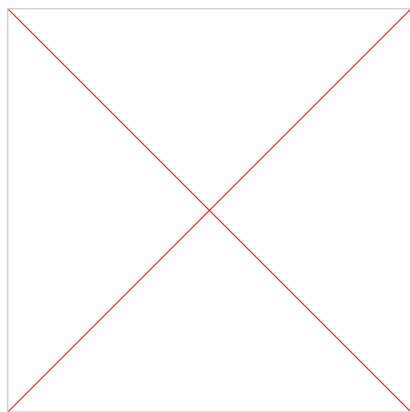
```
py -3.12 -m venv C:\isaacsim-env  
C:\isaacsim-env\Scripts\activate  
pip install torch==2.11.0 --index-url https://download.pytorch.org/whl/cu128  
pip install "isaacsim[all,extscache]==6.0.1.0" --extra-index-url https://pypi.nvidia.com
```

裝好約 **20 GB**: Isaac Sim 15 GB + PyTorch 4 GB, 邊聽邊下載

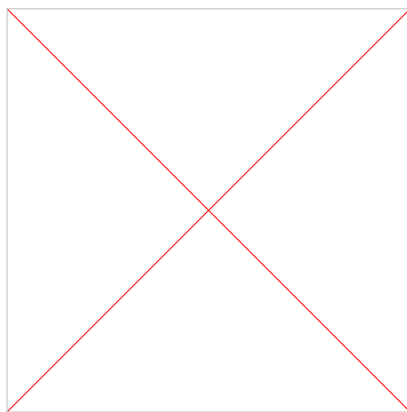
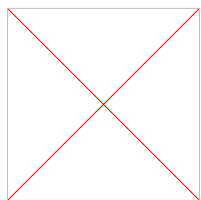
Linux: 前兩行換成

```
python3.12 -m venv ~/isaacsim-env · source ~/isaacsim-env/bin/activate
```

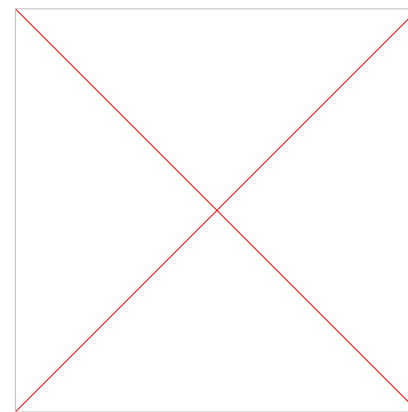
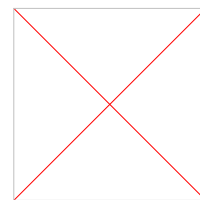
今天怎麼上課



講解



實作



檢查點

實作

看到這個標籤，就是動手時間

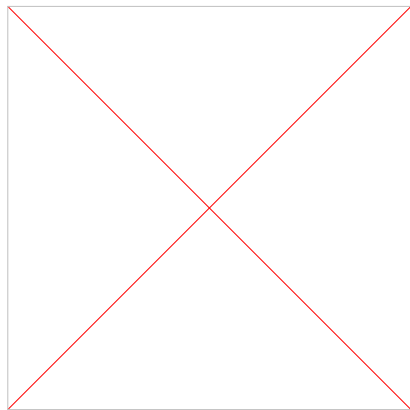
卡住超過 5 分鐘就舉手

Part 1

認識 Isaac

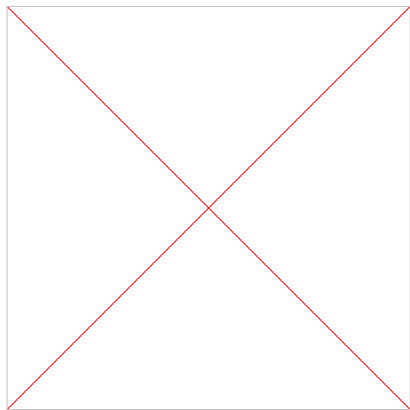
09:00 – 10:00

為什麼要在模擬裡訓練？



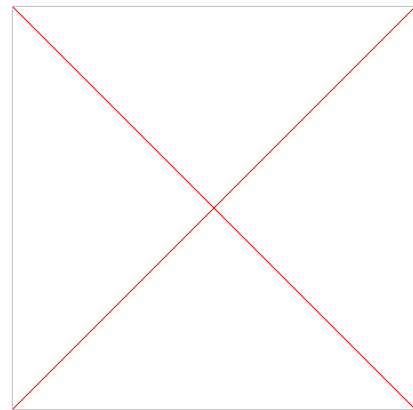
摔不壞

跌倒一萬次
也不用修



可以快轉

比真實時間
快很多倍



一次很多台

同一張 GPU
上千台一起跑

真實 vs 模擬

真實世界

摔一次，修一次

一秒就是一秒

一次只有一台

模擬世界

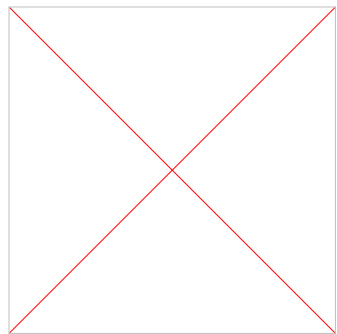
摔一萬次，按重來

可以比真實更快

上千台一起跑

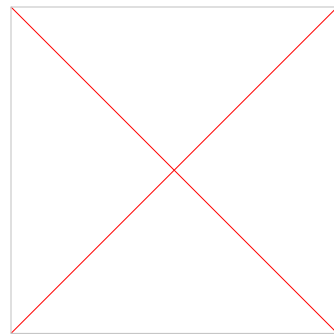
在模擬裡摔一萬次
只為了在現實走好一步

NVIDIA Isaac 家族



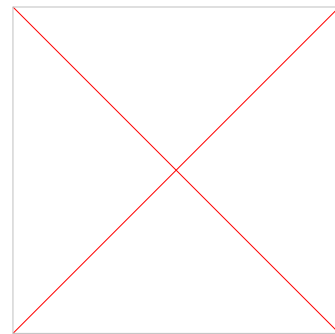
Omniverse

3D 平台
以 USD 為核心



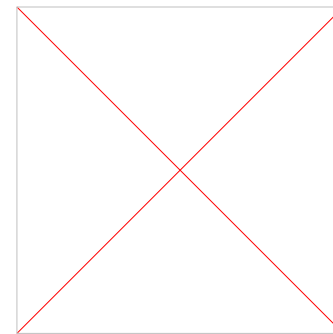
Isaac Sim

機器人模擬器
今天的主角



Isaac Lab

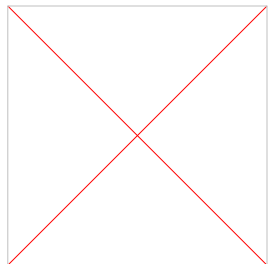
機器人學習框架
平行訓練



Isaac ROS

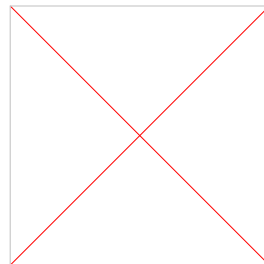
真機上的
ROS 2 加速套件

Isaac Sim 能做什麼？



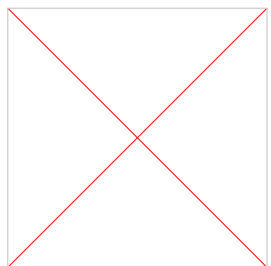
物理 PhysX

剛體、關節、接觸、摩擦



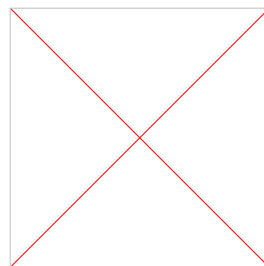
畫面 RTX

即時光線追蹤、合成資料



感測器

相機、光達、IMU



Python

用程式控制一切

Isaac Sim vs Isaac Lab

Isaac Sim

世界本身

場景、物理、感測器

有 GUI 可以直接玩

Isaac Lab

訓練營

上千個平行環境

內建任務與 RL 演算法

今天: Isaac Sim + Python + Stable-Baselines3 (和我們專案的做法一樣)

常見的機器人模擬器

模擬器	特色	常見用途
Gazebo	和 ROS 搭配、生態完整	ROS 系統整合
MuJoCo	輕量、快、接觸計算準	控制、RL 研究
Unity	遊戲引擎、容易上手	互動、視覺化
Isaac Sim	RTX 畫面 + GPU 物理 + 感測器	大規模訓練、合成資料

同一個機器人，兩個物理引擎

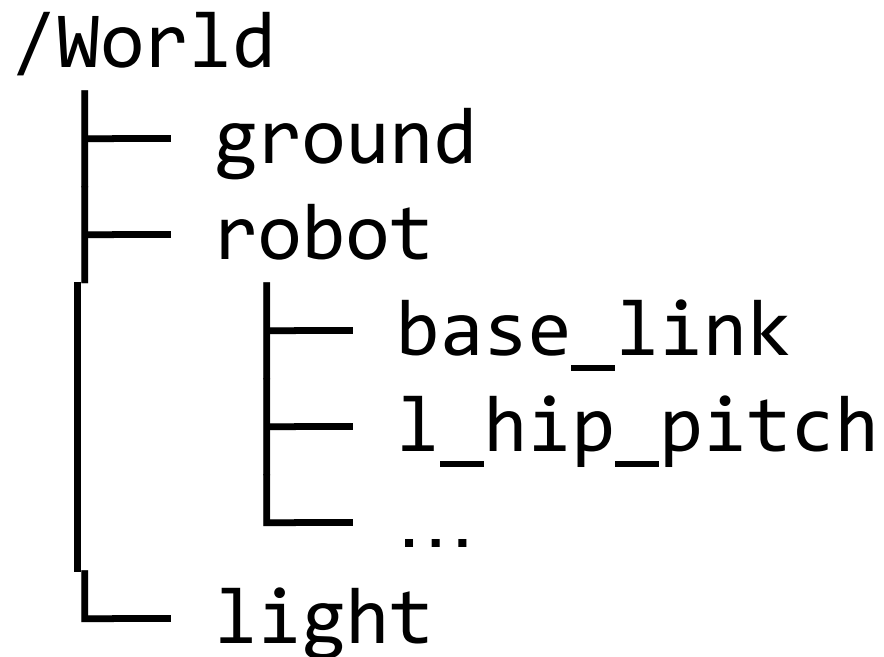
	Unity	Isaac Sim
走 6 步的距離	0.2371 m	0.2392 m
站立高度	0.2362 m	0.2359 m
最大 IK 誤差	0.15 mm	0.150 mm
連走 2000 步	85.5 m · 沒跌倒	83.8 m · 沒跌倒

換了物理引擎，結果幾乎一樣 → 模擬可以相信，但**一定要自己驗證**

三個一定要懂的觀念

USD · 物理步 · 關節驅動

USD: 場景是一棵樹



Stage 整個場景

Prim 樹上的每個節點

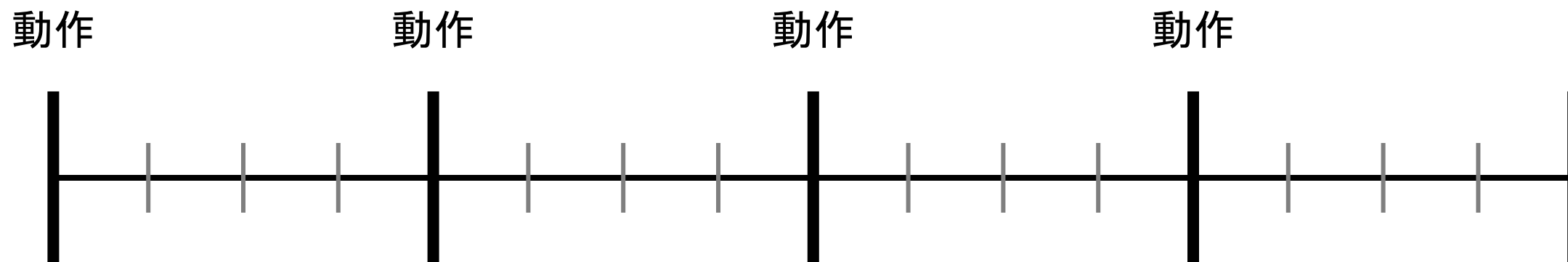
Attribute 節點的屬性

USD 是 Pixar 開發的場景格式

Isaac Sim 裡的一切都是 USD

物理步 vs 控制步

控制 **60 Hz**: 每 16.7 ms 決定一次動作

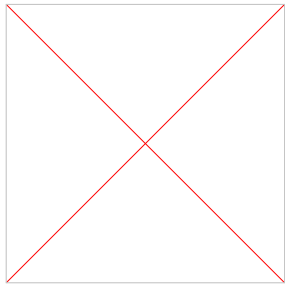


物理 **240 Hz**: 每 4.2 ms 算一次

一個動作 → 物理算 4 次 (decimation = 4)

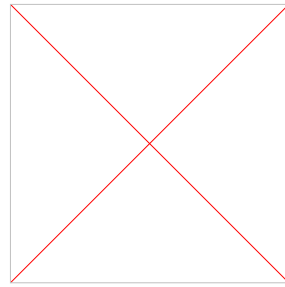
關節怎麼動：PD 控制

$$\text{力矩} = K_p \times (\text{目標角} - \text{目前角}) - K_d \times \text{角速度}$$



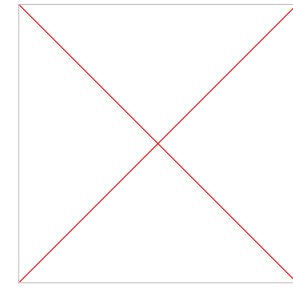
$$K_p = 20$$

像彈簧：拉回目標
N·m / rad



$$K_d = 0.4$$

像阻尼：不要晃
N·m·s / rad

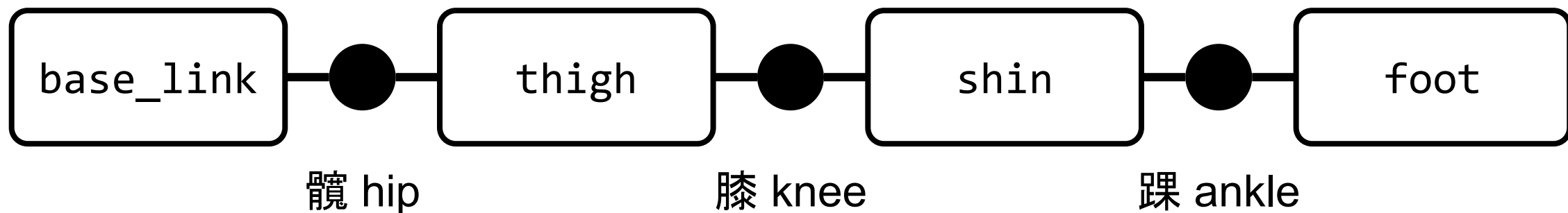


$$0.93 \text{ N}\cdot\text{m}$$

力矩上限
XC330 堵轉力矩

你只給「目標角度」，力矩由 PhysX 幫你算

Articulation: 剛體 + 關節

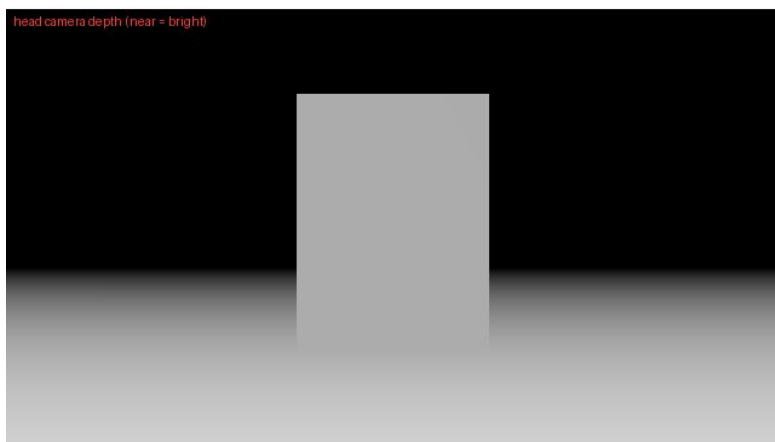
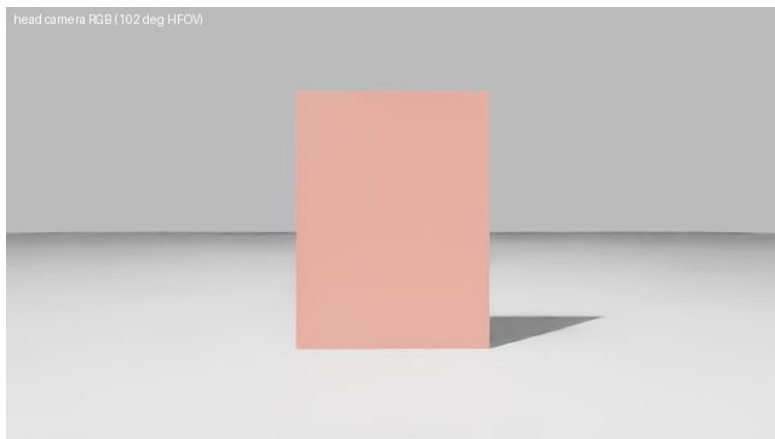


機器人 = 剛體(**link**)用關節(**joint**)連成一棵樹

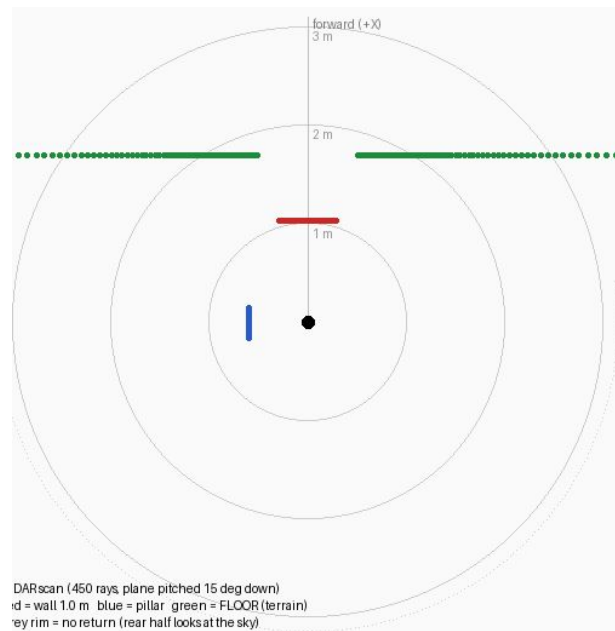
Track C: 腿 $6 + 6$ 、腰 2 、手 $3 + 3 = 20$ 個關節

骨盆在空中可以自由移動: 浮動底座, 再加 6 個自由度

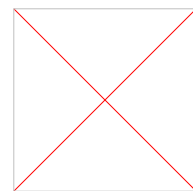
感測器



相機：彩色(上)、深度(下)

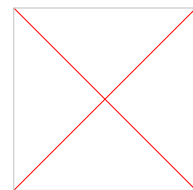


光達：俯視圖



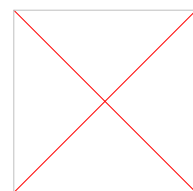
IMU

姿態、角速度、加速度



光達

450 條射線、往下 15°



相機

彩色 + 深度

每一項都對過真值：**15 / 15**

一次跑很多台

9,300

控制步 / 秒

4 個 Isaac 行程 × 512 台機器人 · 一張 RTX 5070 Ti

1000 萬步的 PPO 訓練 ≈ 18 分鐘

示範機器人: Track C



20 個關節

腿 6 + 6 · 腰 2 · 手 3 + 3

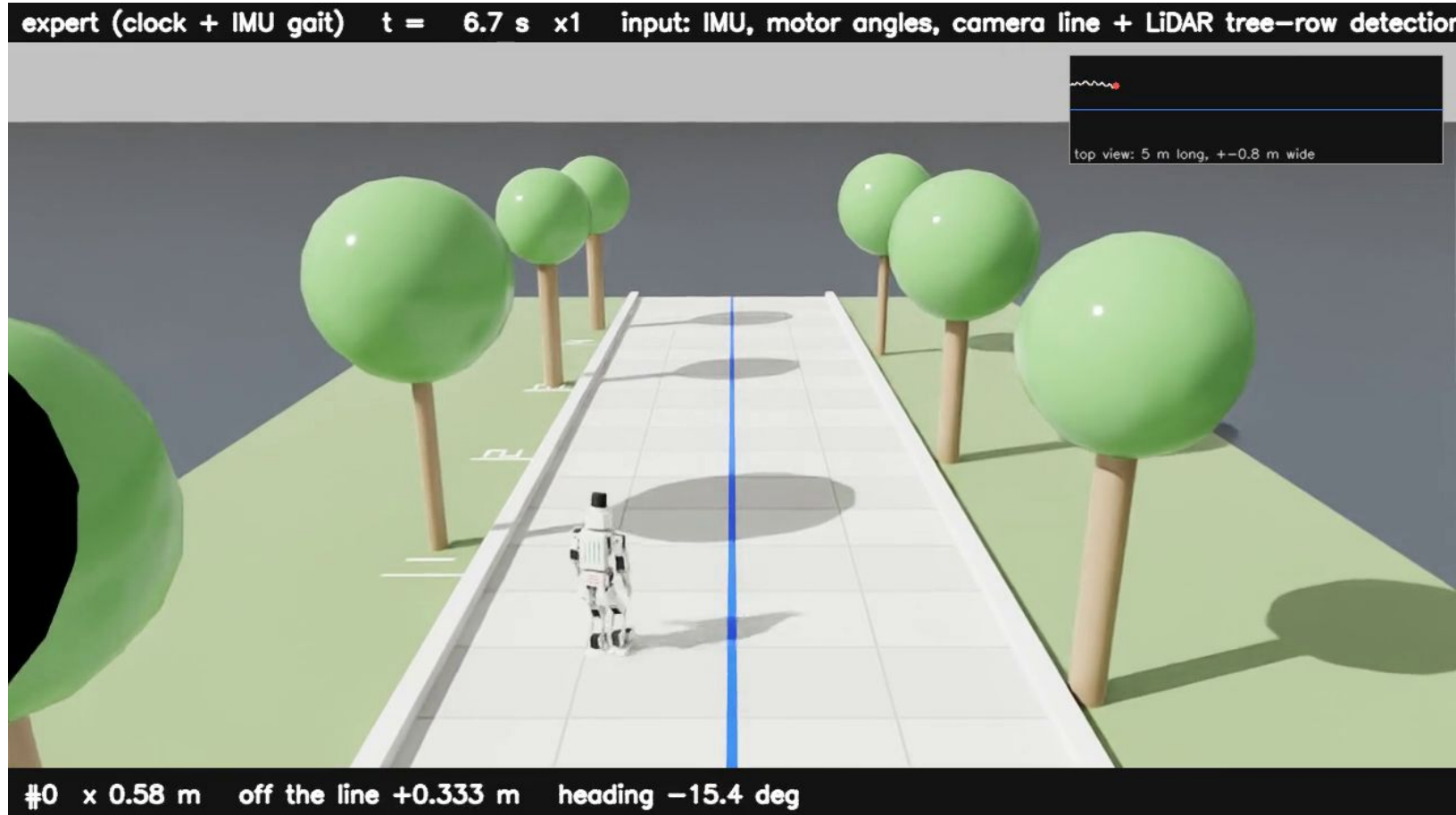
1.17 kg

20 顆 XC330 伺服馬達

3 種感測器

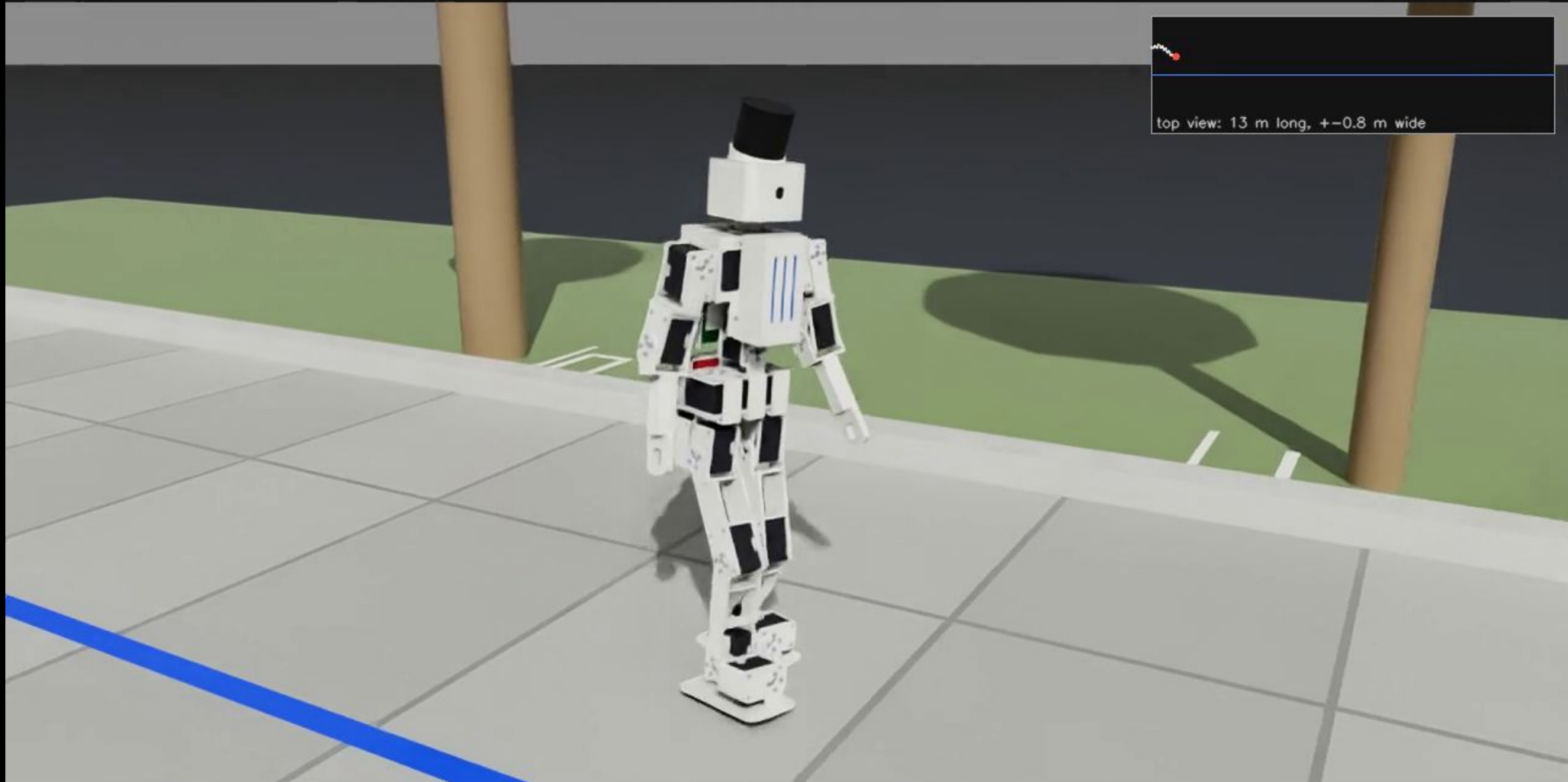
IMU · 光達 · 相機

最後，它走在這樣的路上



相機找地上的線、光達找路樹 → 機器人自己走在人行道中間

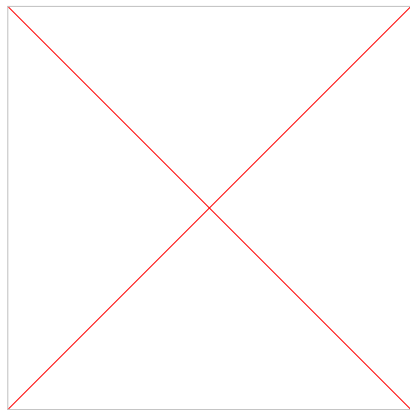
r13 (PPO actor) t = 6.0 s step 15 x1 input: IMU, motor angles, camera line + LiDAR tree-row detection



#0 x 0.78 m off the line +0.231 m heading -18.2 deg

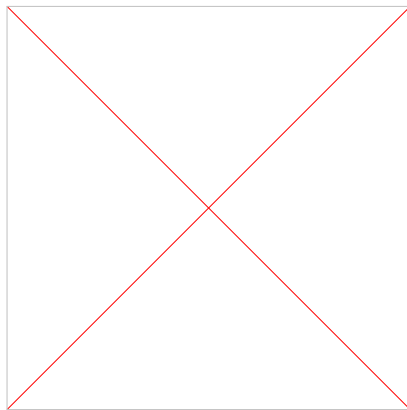
PPO 策略:一開始偏離路線, 自己走回來(Round 3)

Part 1 小結



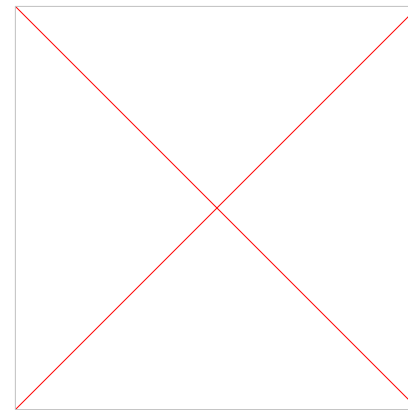
模擬

便宜、快、可以平行



Isaac Sim

物理 + 畫面 + 感測器



三個觀念

USD · 物理步 · PD

Part 2

安裝 Isaac

10:00 – 11:00

官方系統需求 (Isaac Sim 6.0)

項目	最低	建議
作業系統	Windows 11 · Ubuntu 22.04 / 24.04	—
顯示卡	RTX 4080 (16 GB)	RTX 5080 (16 GB)
記憶體	32 GB	64 GB
硬碟	50 GB SSD	500 GB SSD
驅動程式	Windows 595.97 · Linux 595.58.03	最新正式版
Python	3.12	3.12

沒有 RT Core 的卡 (A100、H100) 不支援 · 資料來源: Isaac Sim 6.0 官方文件

但是...

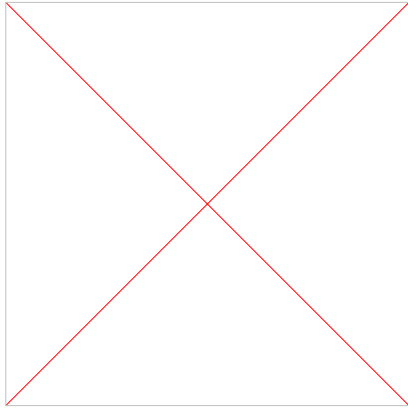
6 GB

RTX 3060 Laptop 也跑得動

單一機器人 · headless · CPU 物理

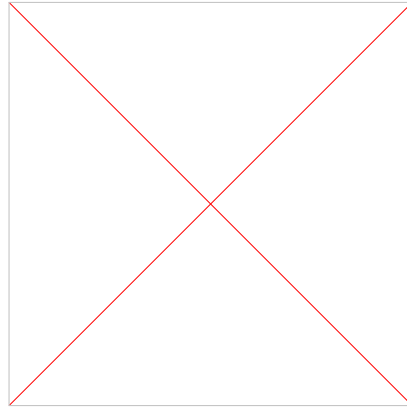
我們另一台人形機器人(Track B)的 Isaac 移植, 就是在這張卡上完成的

沒有 NVIDIA 顯卡怎麼辦？



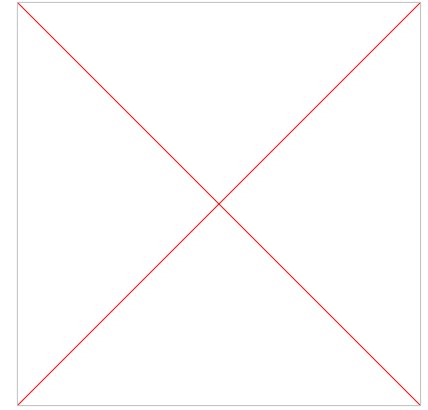
找隊友

和有顯卡的同學
兩人一組



雲端 GPU

要有 RT Core
例如 L4、L40S

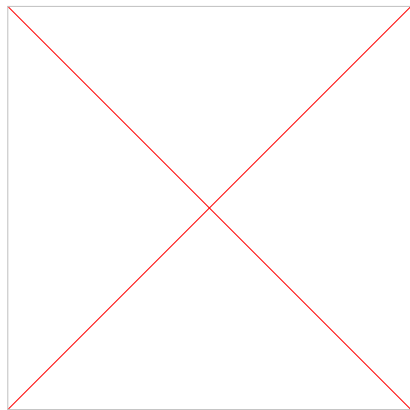


先看懂流程

跟著講者 demo
回家再裝

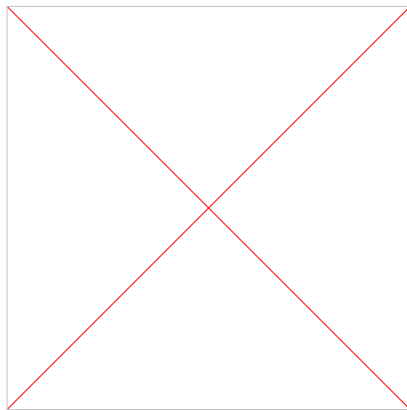
Mac 不支援 Isaac Sim

三種安裝方式



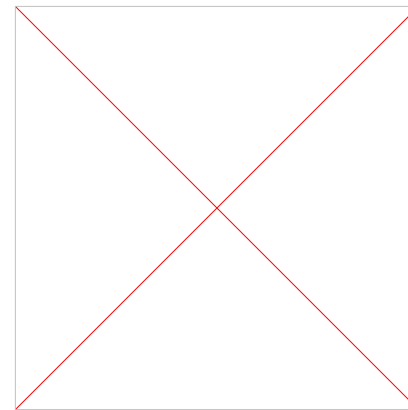
pip

今天用這個
和一般套件一樣裝



Workstation

下載完整安裝包
解壓縮就能用



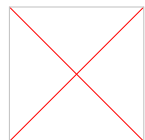
Docker

Linux 伺服器
容器化部署

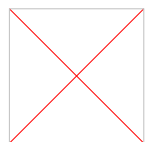
Step 0: 確認顯卡驅動

終端機

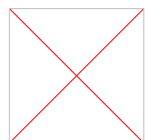
```
nvidia-smi
```



看右上角的 **Driver Version**



Windows $\geq$ **595.97**、Linux $\geq$ **595.58.03**



太舊 → 到 NVIDIA 官網下載新驅動

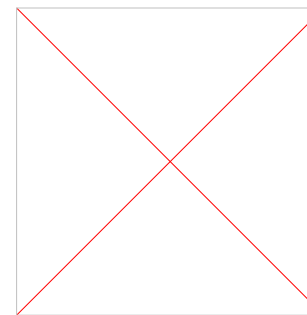
Step 1: 建立 Python 3.12 環境

Windows · PowerShell

```
py -3.12 -m venv C:\isaacsim-env  
C:\isaacsim-env\Scripts\activate  
python -m pip install --upgrade pip
```

Linux

```
python3.12 -m venv ~/isaacsim-env  
source ~/isaacsim-env/bin/activate  
python -m pip install --upgrade pip
```



一定要 3.12

5.x 要 3.11

4.x 要 3.10

版本錯 → pip 找不到

Step 2: 安裝 PyTorch CUDA 版

安裝

```
pip install torch==2.11.0 --index-url https://download.pytorch.org/whl/cu128
```

檢查

```
python -c "import torch; print(torch.cuda.is_available())"
```

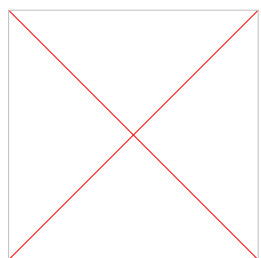
印出 **True** 才對; False → 看 Part 2 最後的常見問題

官方也提供 cu130 版本, 依你的驅動選一個

Step 3 : 安裝 Isaac Sim 6.0.1

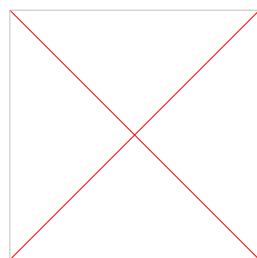
終端機(venv 啟動狀態)

```
pip install "isaacsim[all,extscache]==6.0.1.0" --extra-index-url https://pypi.nvidia.com
```



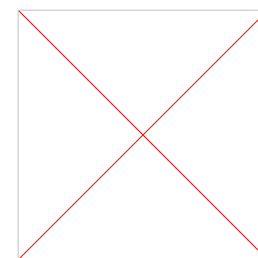
約 15 GB

Isaac Sim 本體
講者實測



Windows

先開長路徑
(下一頁)



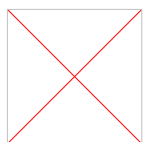
版本要對

Isaac Sim 6.0.1
Python 3.12

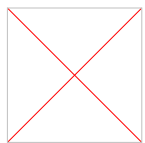
Windows：開啟長路徑支援

PowerShell (以系統管理員身分執行)

```
New-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\FileSystem" `
    -Name LongPathsEnabled -Value 1 -PropertyType DWORD -Force
```



不開的話：pip 解壓縮到一半說「找不到檔案」



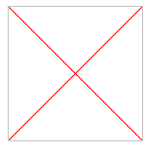
改完要重開終端機

Step 4: 第一次啟動

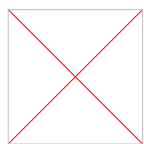
終端機



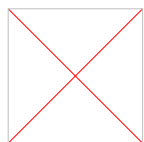
```
isaacsim
```



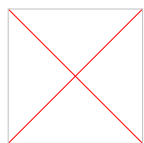
第一次會問 EULA → 輸入 **Yes**



第一次要編譯 shader: 約 **3 分鐘**



之後啟動: 約 **11 秒**

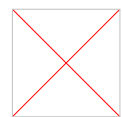


程式裡: OMNI_KIT_ACCEPT_EULA=YES

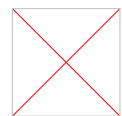
時間是講者 RTX 3060 Laptop 上量到的

安裝 Isaac Sim

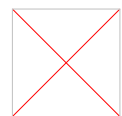
實作 2-1



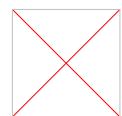
nvidia-smi 看得到驅動版本



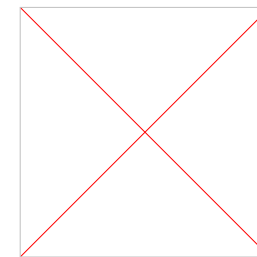
Python 3.12 虛擬環境



torch.cuda.is_available() → True



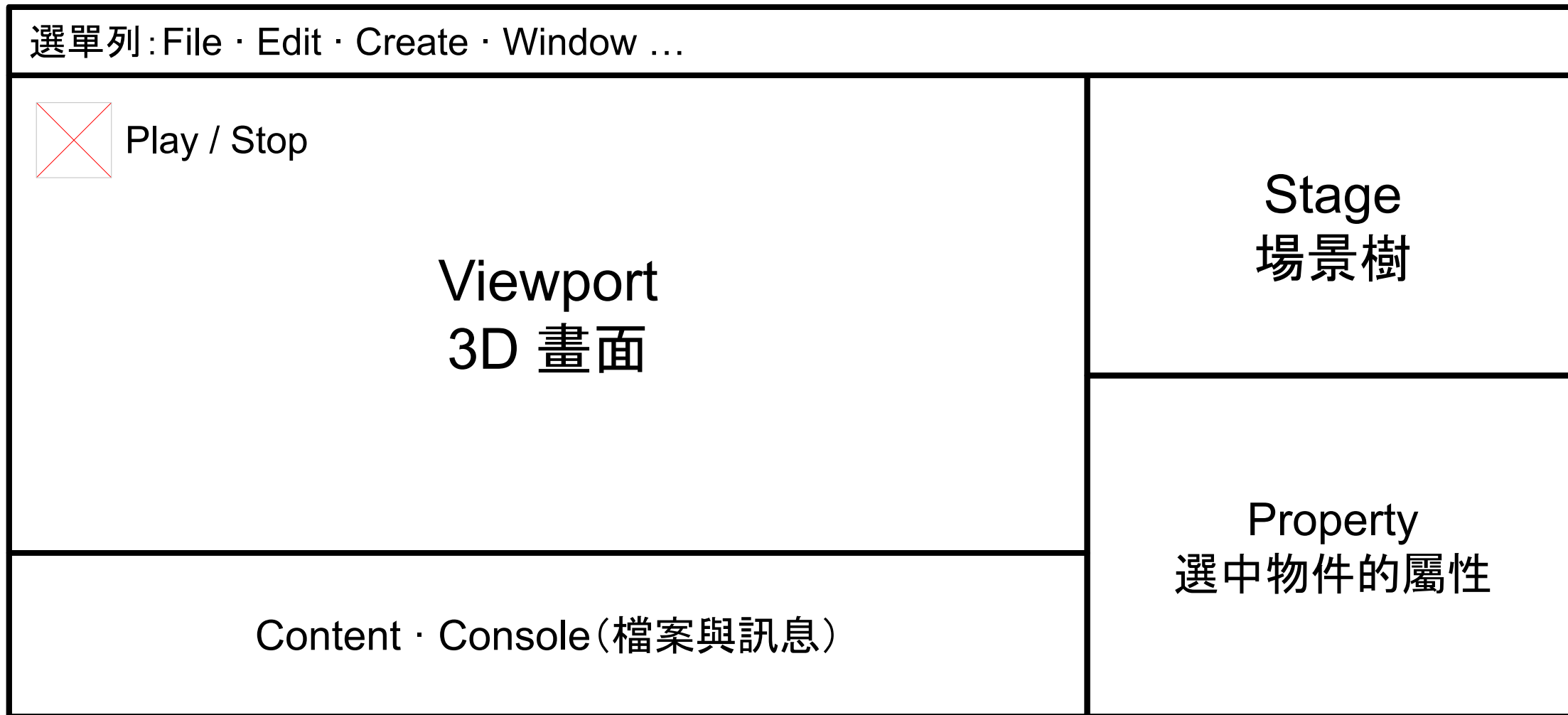
isaacsim 視窗打開了



25
min

等下載的時候, 先看下一頁的介面導覽

Isaac Sim 的介面



第一個場景：用滑鼠

- 1 Create → Physics → Ground Plane
- 2 Create → Shape → Cube
在 Property 把 Translate 的 Z 改成 1
- 3 Cube 按右鍵 → Add → Physics
→ Rigid Body with Colliders Preset
- 4 按 ▶ Play: 方塊掉下來

hello_isaac.py (1/2): 建立世界

```
from isaacsim import SimulationApp
app = SimulationApp({"headless": False})          # 一定要最先建立

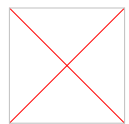
import numpy as np
from isaacsim.core.api import World
from isaacsim.core.api.objects import DynamicCuboid

world = World(stage_units_in_meters=1.0)
world.scene.add_default_ground_plane()            # 預設地板 restitution = 0.8
cube = world.scene.add(DynamicCuboid(
    prim_path="/World/cube", name="cube", size=0.2,
    position=np.array([0.0, 0.0, 1.0]), color=np.array([1.0, 0.0, 0.0])))
world.reset()
```

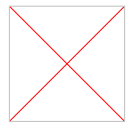
hello_isaac.py (2/2): 模擬迴圈

```
step = 0
while app.is_running():
    world.step(render=True)
    if step % 30 == 0:
        z = cube.get_world_pose()[0][2]
        print(f"step {step:4d}    z = {z:.3f} m", flush=True)
    step += 1
app.close()
```

關掉視窗就結束



world.reset() 之後才能 step

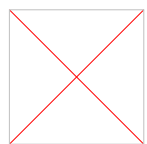


print 要加 flush=True, 不然可能看不到輸出

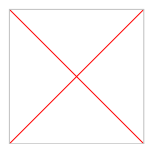
執行看看

終端機

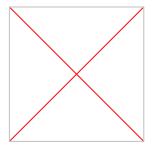
```
python hello_isaac.py
```



紅色方塊從 1 m 高掉下來



.....然後彈了好幾下？



預設地板的 restitution (彈性) = **0.8**

GUI 還是 headless ?

GUI

"headless": False

看得到畫面、比較慢

除錯、展示用

headless

"headless": True

沒有畫面、快很多

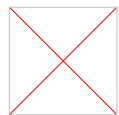
訓練、批次實驗用

安裝常見問題

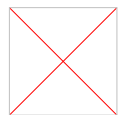
症狀	解法
pip 找不到 isaacsim	確認 Python 是 3.12, 重建 venv
安裝到一半路徑錯誤	Windows 開長路徑, 重開終端機
PowerShell 擋 activate	Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
第一次啟動卡很久	正在編譯 shader, 等 3–5 分鐘
torch.cuda 是 False	重裝 CUDA 版 torch(指令在備忘稿)
print 沒有輸出	print(..., flush=True)
關閉時報錯	Windows 已知問題, 改用 os._exit(0)

hello_isaac.py

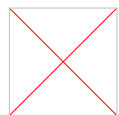
實作 2-2



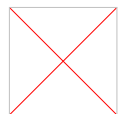
視窗打開，看到紅色方塊



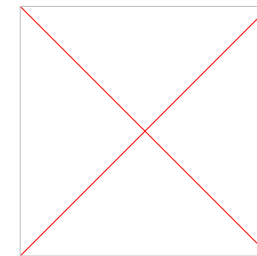
方塊掉到地上(還會彈)



終端機一直印出 z



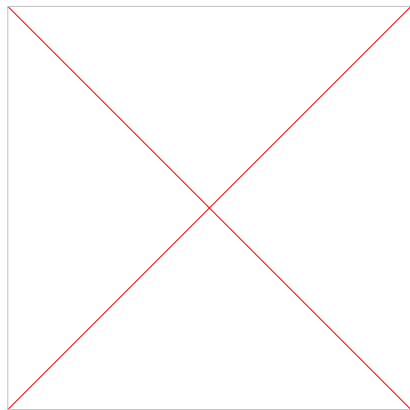
把高度改成 3 m 再跑一次



10
min

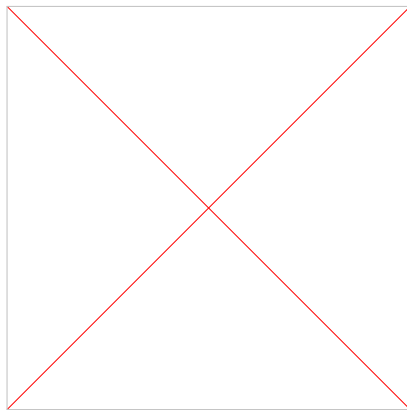
提早做完: 把 size 改大、改顏色、多放幾顆

Part 2 小結



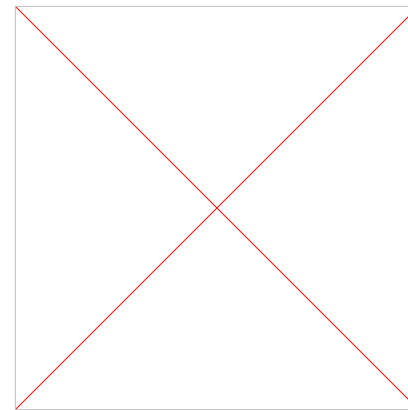
Python 3.12

venv 隔離環境



pip install

isaacsim 6.0.1



SimulationApp

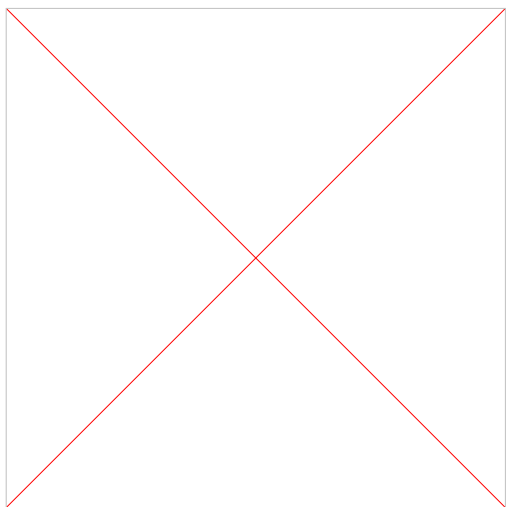
一定要最先建立

Part 3

認識與匯入 URDF

11:00 – 12:00

URDF 是什麼？

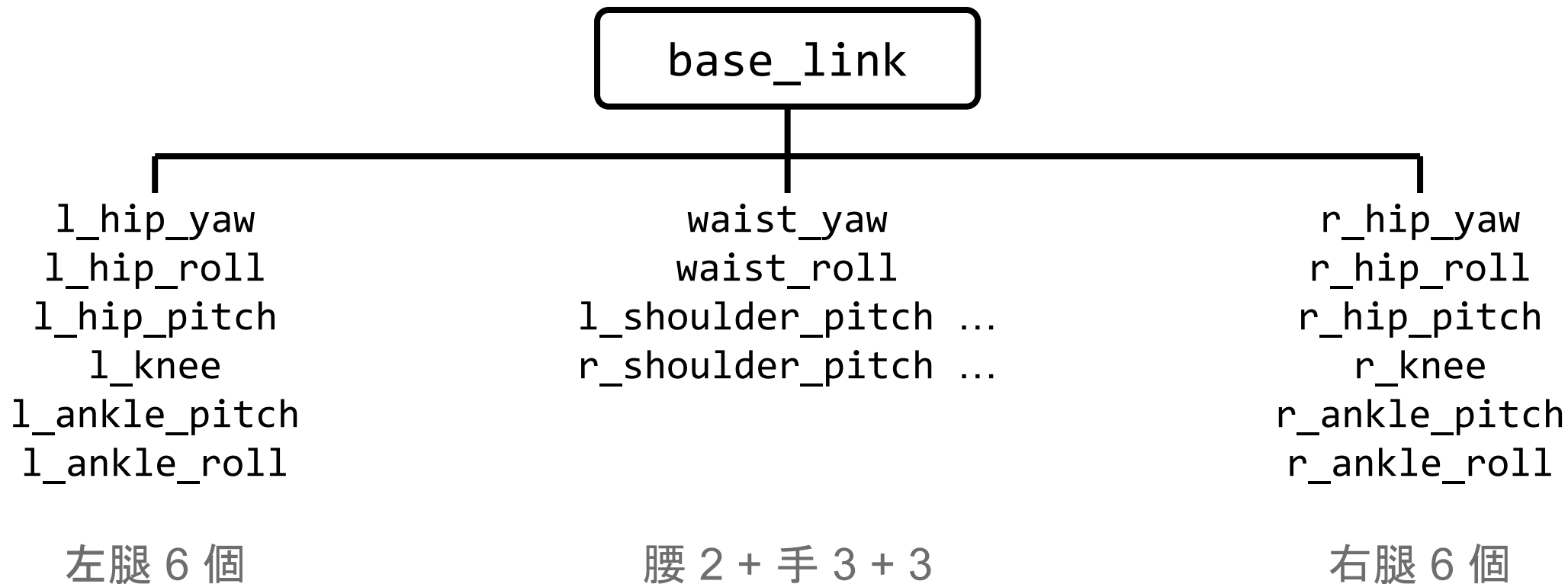


一份 XML 文字檔
描述一台機器人

Unified Robot Description Format

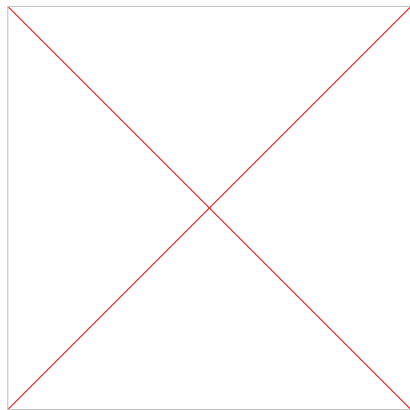
ROS 的標準格式，幾乎每台機器人都有一份

整台機器人是一棵樹



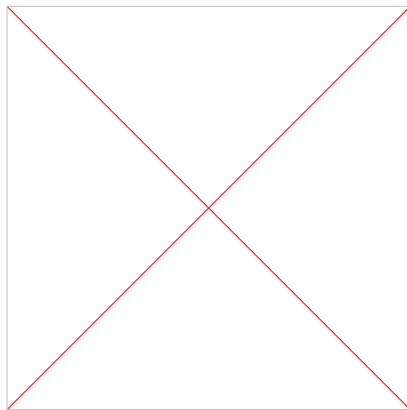
每個 joint 連接一個 parent 和一個 child → 一定是一棵樹

每個 Link 有三個部分



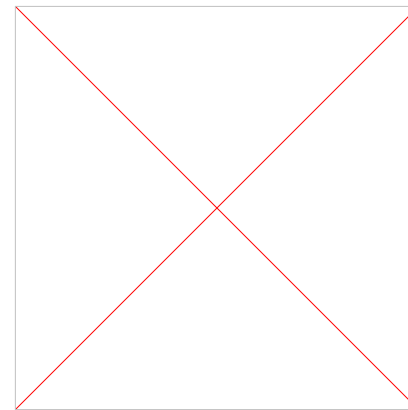
visual

看起來的樣子
網格、顏色



collision

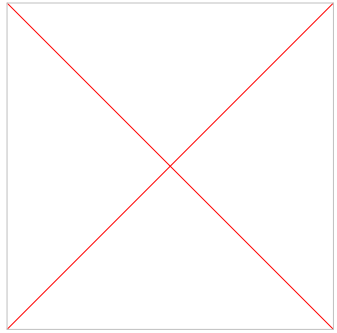
碰撞用的形狀
越簡單越快



inertial

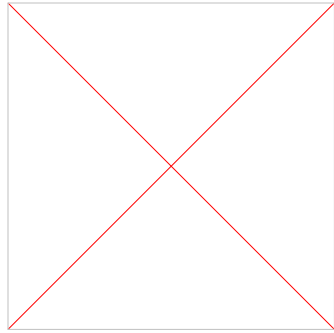
質量、質心、慣量
錯了就站不穩

Joint 的種類



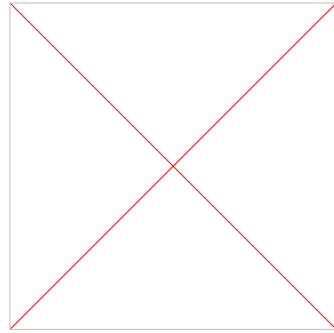
revolute

有範圍的旋轉
手肘、膝蓋



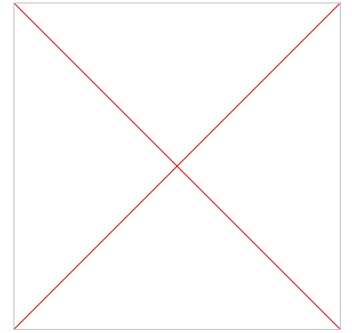
continuous

無限旋轉
輪子



prismatic

直線滑動
升降台



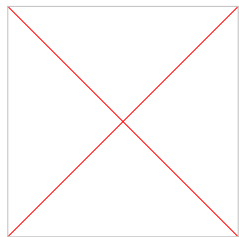
fixed

焊死不動
感測器支架

最小的 URDF

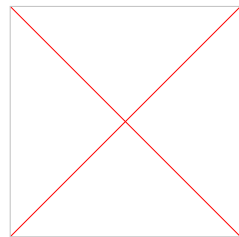
```
<robot name="mini">
  <link name="base_link">
    <visual><geometry><box size="0.1 0.1 0.05"/></geometry></visual>
    <collision><geometry><box size="0.1 0.1 0.05"/></geometry></collision>
    <inertial><mass value="0.2"/>
      <inertia ixx="2e-4" iyy="2e-4" izz="3e-4" ixy="0" ixz="0" iyz="0"/></inertial>
    </link>
  <link name="arm"> <!-- 一樣有 visual / collision / inertial --> </link>
  <joint name="shoulder" type="revolute">
    <parent link="base_link"/> <child link="arm"/>
    <origin xyz="0 0 0.05" rpy="0 0 0"/>
    <axis xyz="0 1 0"/>
    <limit lower="-1.57" upper="1.57" effort="0.9" velocity="5"/>
  </joint>
</robot>
```

Joint 的四個關鍵



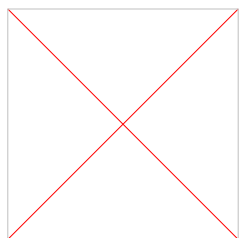
parent / child

誰接誰：決定樹的方向



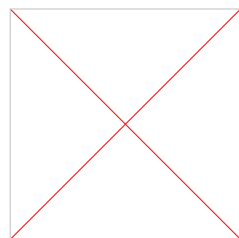
origin

關節在 parent 的哪裡、怎麼擺



axis

繞哪個軸轉(右手定則)



limit

角度範圍、力矩、速度上限

Track C 的膝蓋: `lower="-0.087"` `upper="2.007"` `effort="0.93"` `velocity="8.48"`

單位：永遠是公制

m

長度：公尺

kg

質量：公斤

rad

角度：弧度

陷阱：匯入後，USD 裡的關節限位單位是「度」

很小的數字不一定是錯的：2.4 cm 的機器人，0.006 就是 6 mm

Track C 的 URDF



21 個 link

零件(含骨盆)

20 個 joint

全部 revolute

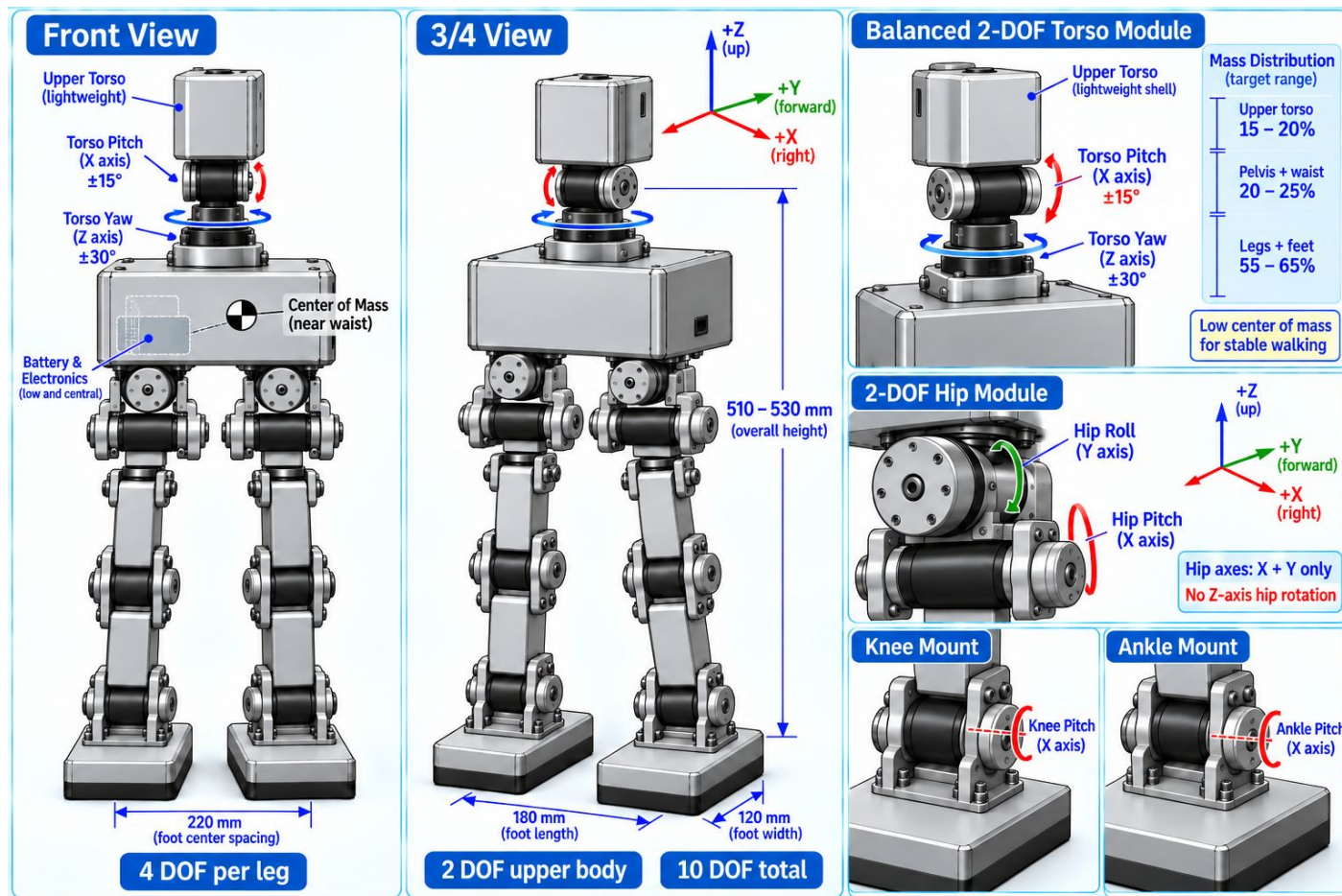
1167.5 g

URDF 與 USD 完全一致

14 個碰撞盒

沒有用網格當碰撞

你們的機器人: Biped_Robot_Master



外觀設計圖(Drive 資料夾裡的圖)

29 kg

兩隻腳就佔 12 kg

10 個關節

每腿 4 + 腰 2, 沒有踝 roll

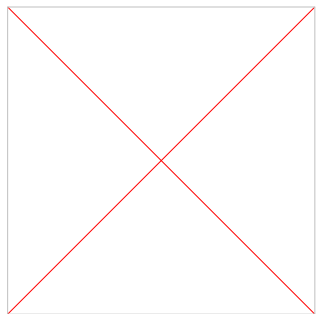
19 cm

重心高度(很低)

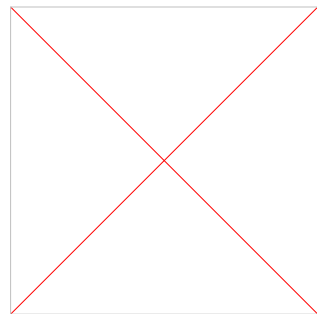
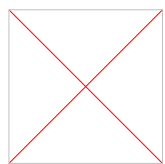
+Y 是前方

+X 是右邊

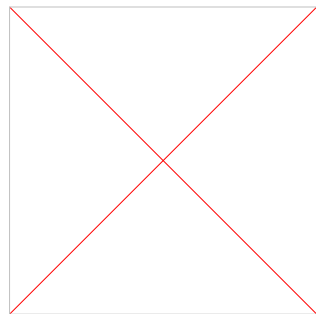
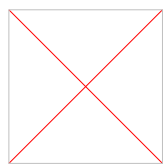
URDF $\rightarrow$ USD



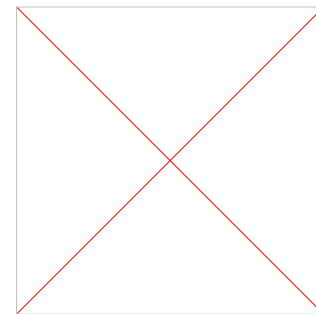
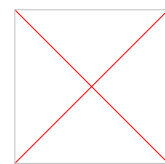
xxx.urdf



URDF Importer



xxx.usda



Isaac Sim

Isaac Sim 的世界是 USD $\rightarrow$ URDF 要先轉換

轉一次就好：之後每次都直接載入 .usda

方法一：用滑鼠匯入

- 1 File → Import
- 2 選你的 .urdf 檔
URDF 要和旁邊的 ..._description/meshes 放在一起
- 3 設定匯入選項(下一頁)
- 4 按 Import: 機器人出現在場景裡

匯入選項怎麼選

選項	設定	為什麼
fix_base	False	會走路:底座不能釘死
collision_from_visuals	False	用 URDF 自己的碰撞形狀
collision_type	凸包	網格碰撞 → Convex Hull
allow_self_collision	True	PhysX 預設就會自碰撞
link_density	None	質量用 URDF 的, 不要自己算
merge_fixed_joints	False	保留每個零件, 方便檢查

fix_base 有三種值: True 釘死 · False 浮動 · None 照原檔

方法二:import_urdf.py (1/2)

```
urdfs = sys.argv[1:] or glob.glob("**/*.urdf", recursive=True)
if len(urdfs) != 1:
    print("\n".join(urdfs))
    sys.exit(f"找到 {len(urdfs)} 個 .urdf:請用 python import_urdf.py <檔案> 指定一個")
urdf = os.path.abspath(urdfs[0])
name = os.path.splitext(os.path.basename(urdf))[0]
shutil.rmtree(os.path.join("usd", name), ignore_errors=True) # 不刪的話第二次會變 xxx_01

from isaacsim import SimulationApp
app = SimulationApp({"headless": True})

from isaacsim.core.utils.extensions import enable_extension
enable_extension("isaacsim.asset.importer.urdf")
app.update()
from isaacsim.asset.importer.urdf import URDFImporter, URDFImporterConfig
```

方法二:import_urdf.py (2/2)

```
cfg = URDFImporterConfig(
    urdf_path=urdf,
    usd_path=os.path.abspath("usd"), # 輸出資料夾 → usd/<名字>/<名字>.usda
    fix_base=False, # 會走路:底座不固定
    merge_fixed_joints=False,
    collision_from_visuals=False, # 用 URDF 自己的 collision
    collision_type="Convex Hull", # 網格碰撞 → 凸包
    allow_self_collision=True, # 6.0.1 只寫給 Newton;PhysX 預設就會自碰撞
    link_density=None, # 質量用 URDF 的, 不要自己算
    run_asset_transformer=False, # 輸出單一 .usda
)
usd = URDFImporter(cfg).import_urdf()
with open("robot_usd.txt", "w") as f: # 記下來:stand / train / play 預設讀它
    f.write(usd)

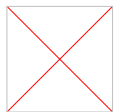
print("wrote", usd, flush=True)
os._exit(0) # Windows 上 app.close() 可能報錯, 直接結束
```

注意: 6.0 換了匯入 API

Isaac Sim 5.x 的寫法	6.0
<code>from isaacsim.asset.importer.urdf import _urdf</code>	已移除
URDFCreateImportConfig 指令	不存在
URDFParseAndImportFile 指令	不存在

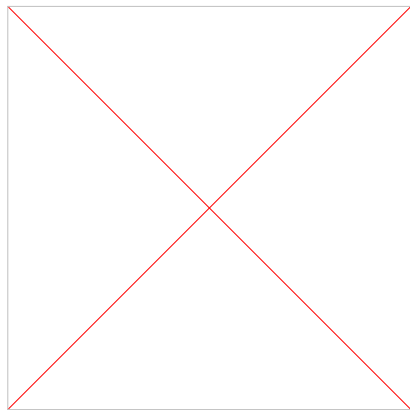
6.0 的寫法

```
URDFImporter(URDFImporterConfig(urdf_path=..., usd_path=...)).import_urdf()
```



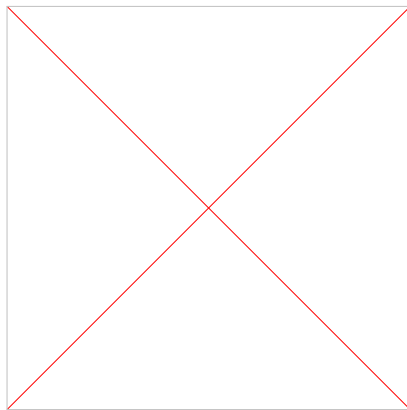
網路上的舊教學、AI 寫的程式，常常還是 5.x 的寫法

匯入後一定要檢查



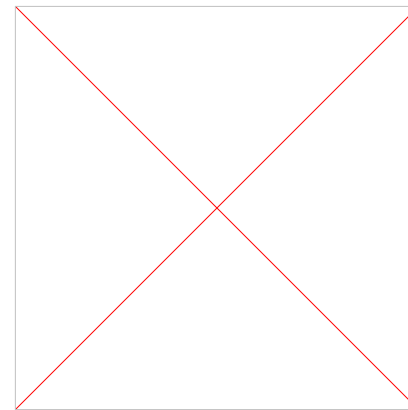
質量

每個 link 的
質量與質心



限位

每個 joint 的
角度範圍



碰撞形狀

盒子或凸包
不能是凹網格

我們的做法：重新讀 USD, 和 URDF 逐項對答案

我們的檢查結果

convert_urdf.py 的輸出 (Track C)

```
total mass: urdf 1167.5 g    usd 1167.5 g
collision prims: 14 cube, 0 mesh, 0 other
=====
OK -- 21 links, 20 joints, masses and limits match the URDF
=====
```

匯入器可能偷偷：用網格重算慣量、丟掉限位、把盒子變凸包

→ 都不會報錯，只會讓機器人「站得低 4 mm」

陷阱 1：機器人彈飛了？

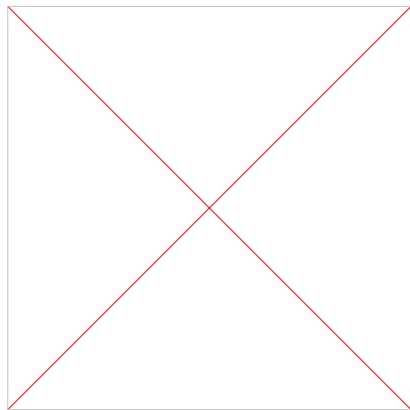
0.8

預設地板的
restitution

```
mat = PhysicsMaterial(prim_path="/World/mat",  
                      static_friction=0.9,  
                      dynamic_friction=0.8,  
                      restitution=0.0)  
GroundPlane(prim_path="/World/ground", size=50,  
            physics_material=mat)
```

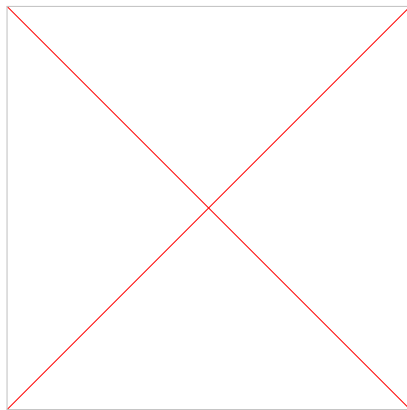
2 mm 的掉落就彈起來、跌倒 → 看起來像感測器壞了

陷阱 2：機器人放哪裡



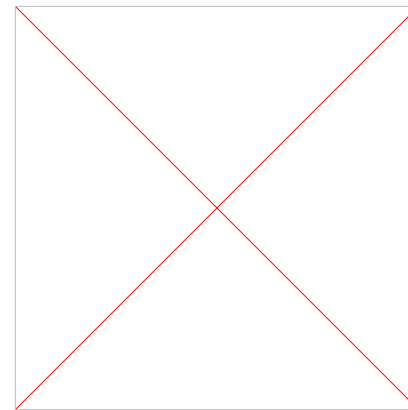
離地 2 mm

太高會摔
太低會卡進地板



建立時就放好

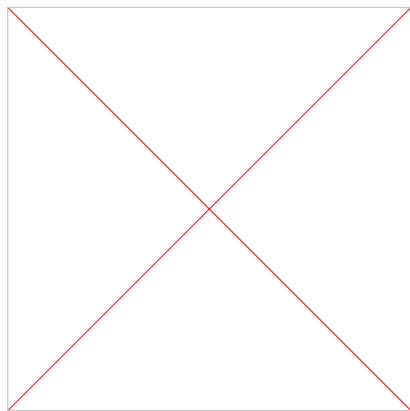
reset 後才瞬移
會互撞飛走



瞬移後速度歸零

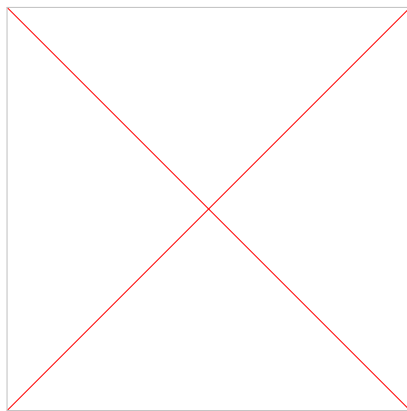
不然會帶著
舊的速度

陷阱 3: 換一台機器人, 數字要重新量



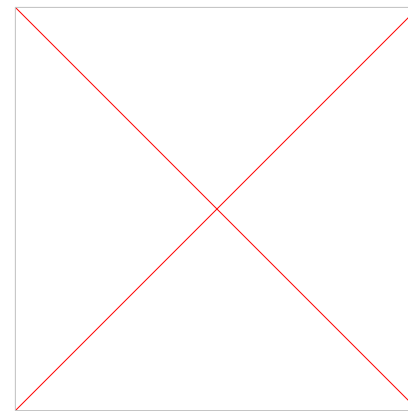
前方是 +Y

腳尖往 +Y 伸 125 mm
腳跟往 -Y 只有 55 mm



base_link 在腳底

原點比腳底低 4.8 mm
不能用高度判斷跌倒



29 kg

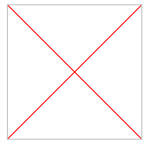
Kp 600、Kd 20
Track C 1.2 kg:20 / 0.4

0925 版在零度姿勢量到的數字(程式裡對應 FWD、跌倒判斷、KP / KD)

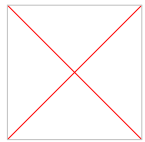
陷阱 4：關節名稱會被改掉

0925 版實測：URDF 的名字 → USD 的名字

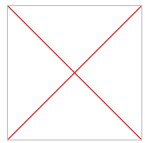
"Rigid 41" → "tn__Rigid41_i7"



USD 的名字不能有空格：匯入時會被「編碼」改名



舊版的 Revolute 15 → tn__Revolute15_oA



找關節用 (parent, child) 配對，或印出 dof_names 看

stand.py (1/3): 地板和燈

```
USD = sys.argv[1] if len(sys.argv) > 1 else open("robot_usd.txt").read().strip()
KP, KD = 600.0, 20.0 # 29 kg:600 / 20(Track C 1.2 kg:20 / 0.4)
world = World(stage_units_in_meters=1.0, physics_dt=1 / 240, rendering_dt=1 / 60)
mat = PhysicsMaterial(prim_path="/World/mat", static_friction=0.9,
                      dynamic_friction=0.8, restitution=0.0) # 地板不要彈
GroundPlane(prim_path="/World/ground", size=50, physics_material=mat)
stage = omni.usd.get_context().get_stage()
UsdLux.DomeLight.Define(stage, "/World/light").CreateIntensityAttr(1000.0) # 沒燈會全黑
set_camera_view(eye=[1.2, 1.6, 0.8], target=[0.0, 0.0, 0.25]) # 鏡頭拉近
```

stand.py (2/3): 把機器人放好

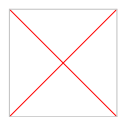
```
prim = add_reference_to_stage(usd_path=USD, prim_path="/World/robot")
box = UsdGeom.BBoxCache(Usd.TimeCode.Default(), [UsdGeom.Tokens.default_])
z_min = box.ComputeWorldBound(prim).ComputeAlignedRange().GetMin()[2]
UsdGeom.XformCommonAPI(prim).SetTranslate(Gf.Vec3d(0, 0, -z_min + 0.002))
root = next(str(p.GetPath()) for p in stage.Traverse()
            if p.HasAPI(UsdPhysics.ArticulationRootAPI))
robot = Articulation(prim_paths_expr=root, name="robot")
world.scene.add(robot)
world.reset()
```

stand.py (3/3): 用 PD 撐住

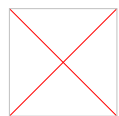
```
n = robot.num_dof
robot.set_gains(kps=np.full((1, n), KP, np.float32),
                kds=np.full((1, n), KD, np.float32))
q0 = np.zeros((1, n), np.float32)
print("關節:", robot.dof_names, flush=True)
step = 0
while app.is_running():
    robot.set_joint_position_targets(q0)
    for _ in range(4):                # 物理 240 Hz × 4 = 控制 60 Hz
        world.step(render=False)
    world.render()                    # 畫面更新一次(不推進物理)
    if step % 60 == 0:
        w, x, y, z = robot.get_world_poses()[1][0]
        tilt = np.degrees(np.arccos(np.clip(1 - 2 * (x * x + y * y), -1, 1)))
        print(f"t = {step / 60:5.1f} s    身體傾斜 = {tilt:4.1f}°", flush=True)
    step += 1
```

匯入你的 URDF

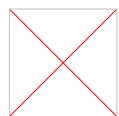
實作 3-1



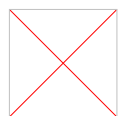
下載 0925 版的整個資料夾



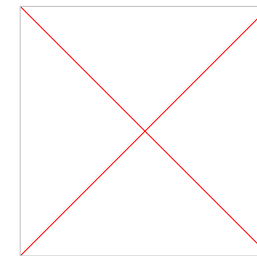
`python import_urdf.py`



多出 `robot_usd.txt` 和 `usd/`



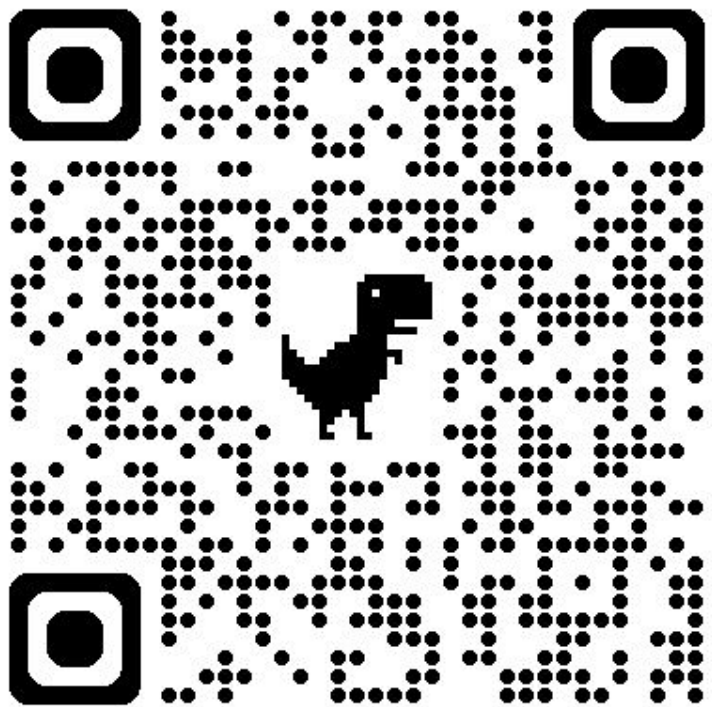
用 `isaacsim` 打開 `.usda` 看看



15
min

資料夾路徑不要有中文或空格

URDF 連結

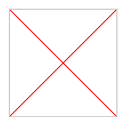


go.ncku.net/1ZY8Hd

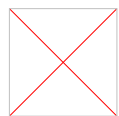
- 1 打開「雙足機器人_髖關節連接件修正」
最新版 0925
- 2 下載整個 ..._unity_urdf 資料夾
Biped_Robot_Master_0925_unity_urdf
- 3 解壓縮到 workshop 資料夾
路徑不要有中文或空格

讓它站起來

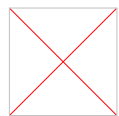
實作 3-2



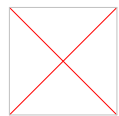
python stand.py



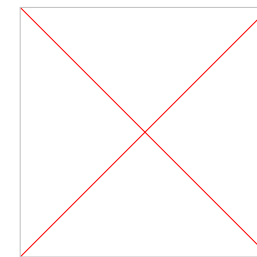
沒有彈飛、沒有穿進地板



站 10 秒，傾斜小於 5°



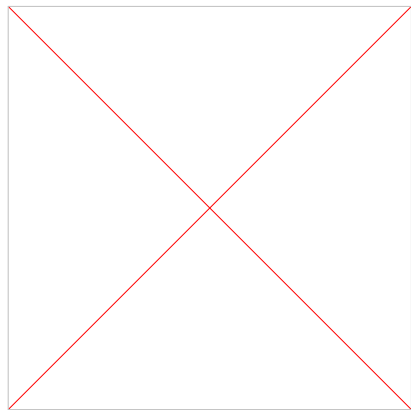
KP 改成 60、再改成 6，比較看看



15
min

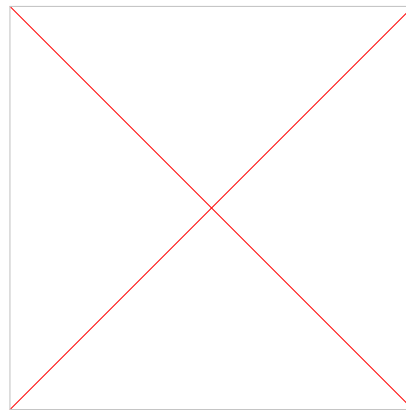
倒了？先查：地板彈性 → 生成高度 → PD 增益

進階：確認關節方向



左右鏡像

左右給對稱的指令
位置誤差 0.000 mm

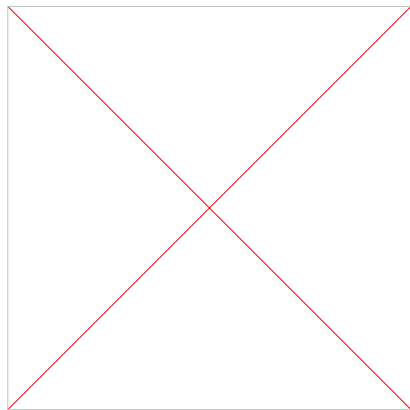


膝蓋方向

膝蓋彎 → 腳離骨盆變近
230 → 139 mm

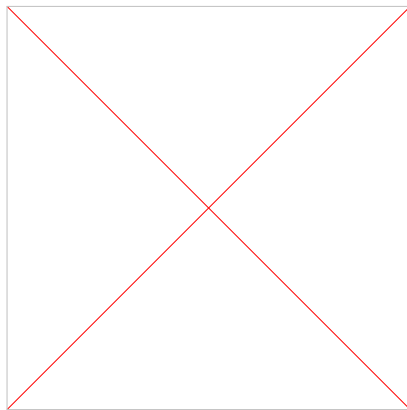
只讀 axis 欄位證明不了什麼：要讓它動起來量

Part 3 小結



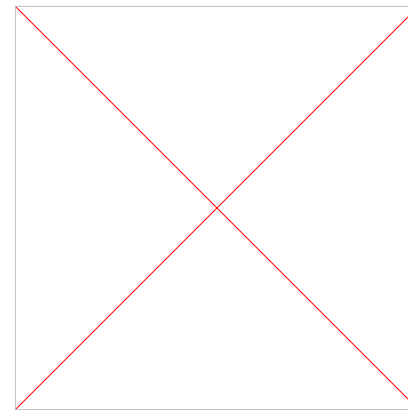
URDF

link + joint 的樹



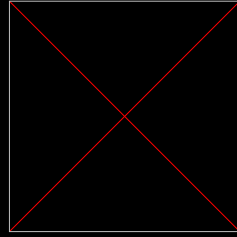
URDFImporter

Isaac Sim 6.0 的新 API



一定要驗證

質量 · 限位 · 碰撞



LUNCH

12:00 – 13:00

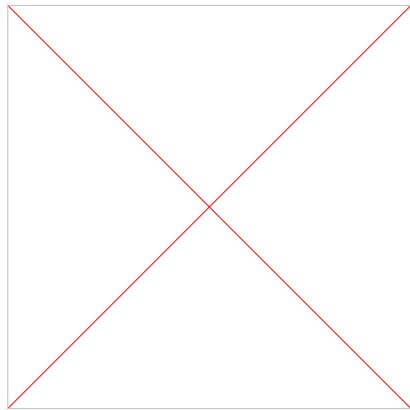
下午：讓機器人學會走

Part 4

強化學習

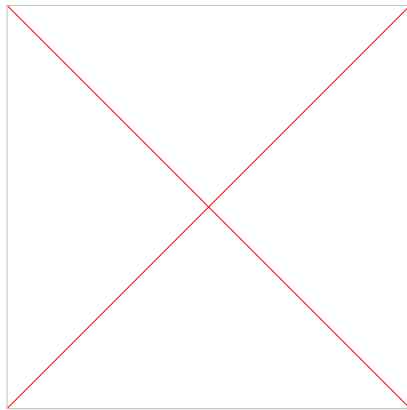
13:00 – 14:00

怎麼讓機器人走路？



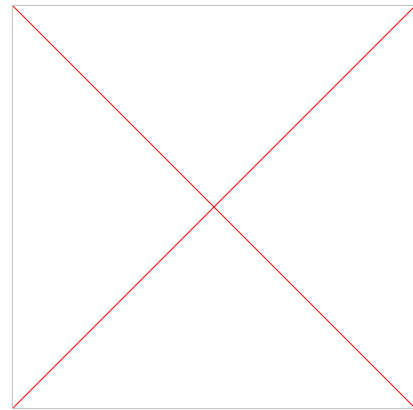
Rule base

工程師手寫步態



MLP

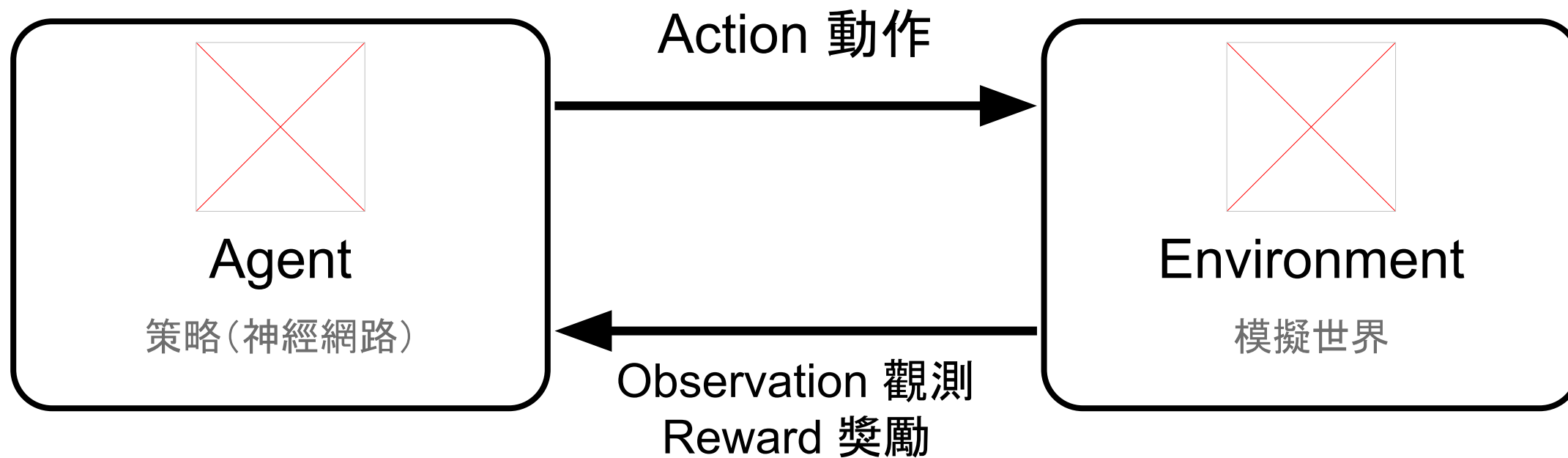
模仿專家的動作



PPO

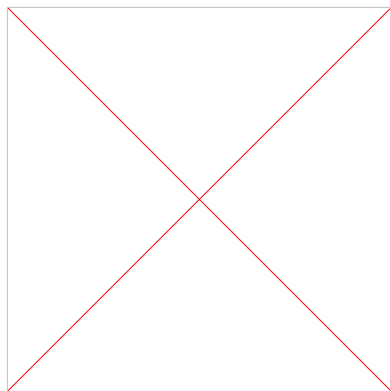
自己從嘗試中學

強化學習的迴圈

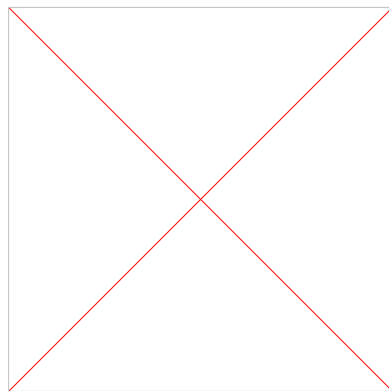
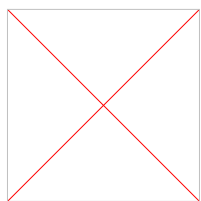


每 1/60 秒轉一圈: 看 → 做 → 得分

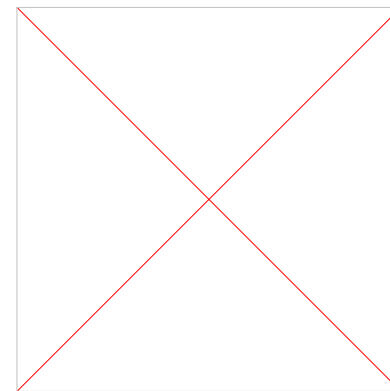
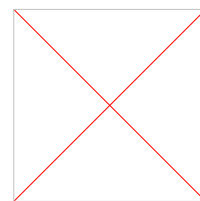
像訓練狗狗



做一個動作



給獎勵



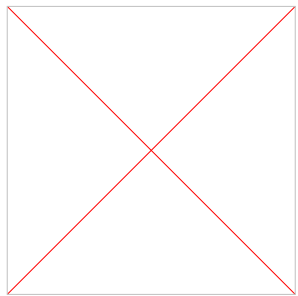
下次更常做

獎勵只說「好不好」，不說「怎麼做」

五個名詞

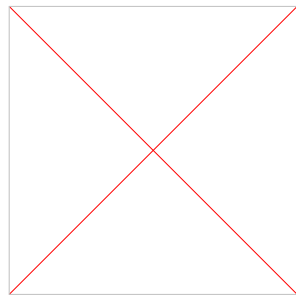
名詞	意思	在我們的機器人
Observation	看到什麼	IMU、馬達角度
Action	做什麼	每個關節的目標角度
Reward	做得好不好	活著、往前、站直
Policy	看到 → 做什麼	一個小神經網路
Episode	一回合	從開始到跌倒或時間到

機器人看到什麼？



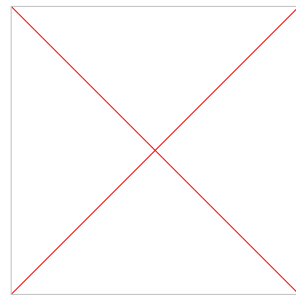
IMU

重力方向
角速度、加速度



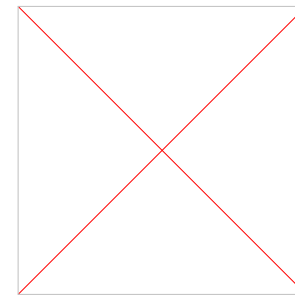
馬達角度

20 個關節
現在的角度



上一拍指令

自己剛剛
做了什麼



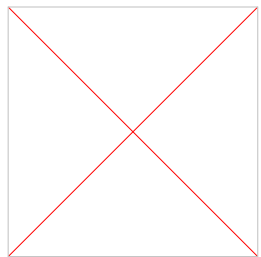
步態時鐘

$\sin / \cos$
現在走到哪

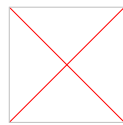
原則：只放**真機量得到**的東西

位置、速度的真值只能拿來算獎勵，不能當觀測

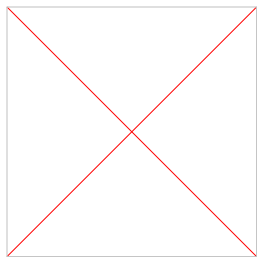
Action: 機器人怎麼動？



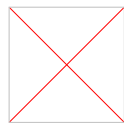
神經網路



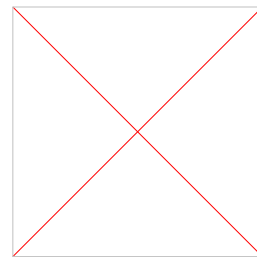
20 個目標角



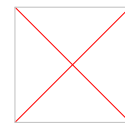
PD 控制



力矩



物理模擬



動作 = 相對預設蹲姿的偏移 ($\text{yaw} / \text{roll} \pm 0.50 \text{ rad}$, 其他 $\pm 0.90 \text{ rad}$)

網路只負責「要轉到哪」, 「用多大力」交給 PD

獎勵怎麼給（我們的設定）

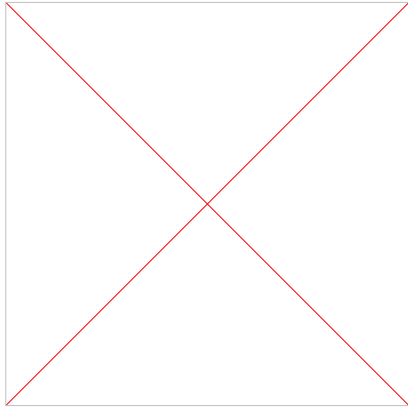
項目	權重	意思
活著	+1.0	每一步沒倒就加分
前進速度	+2.0	越接近目標速度越高
身體直立	+0.5	不要歪
側向偏移	-1.0	不要往旁邊飄
轉向速度	-0.2	不要一直轉
動作變化	-0.02	動作不要抖
跌倒	-50	一次扣很多，然後結束

Track C 的 contract_c.py · 前進速度用高斯形狀 ($\sigma = 0.05$ m/s)

AI 很會鑽漏洞

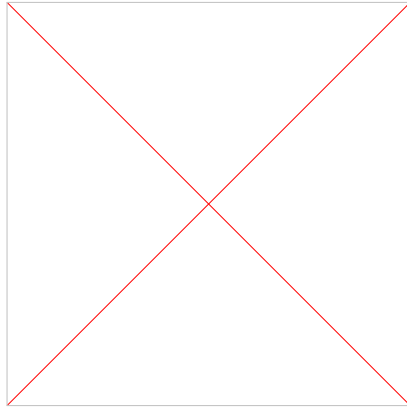
Reward Hacking

真實案例：它學到的不是你想的



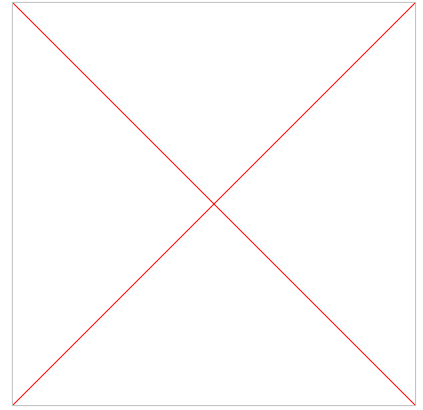
原地踏步

前進獎勵太小
站著最安全



停在樓梯前

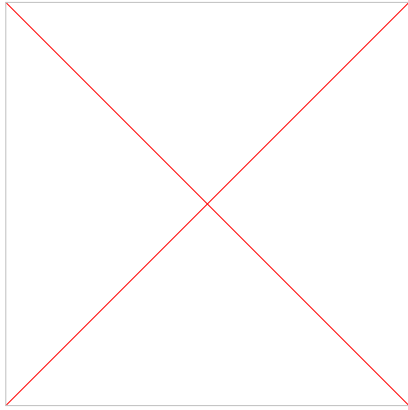
只看存活率
不爬最安全



全體繞圈

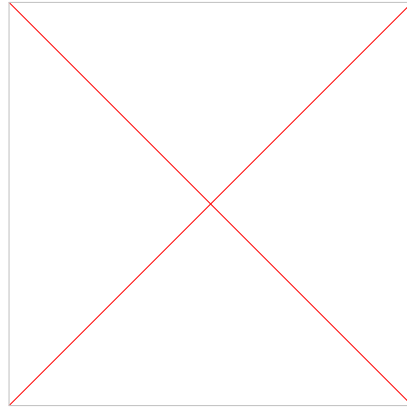
看不到航向
一起往同一邊彎

什麼時候算結束？



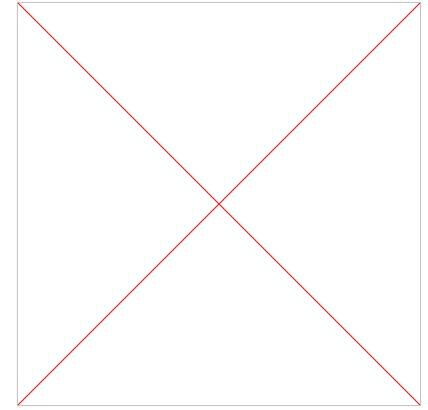
傾斜 $> 45^\circ$

跌倒：扣分並結束



骨盆 $< 0.15\text{ m}$

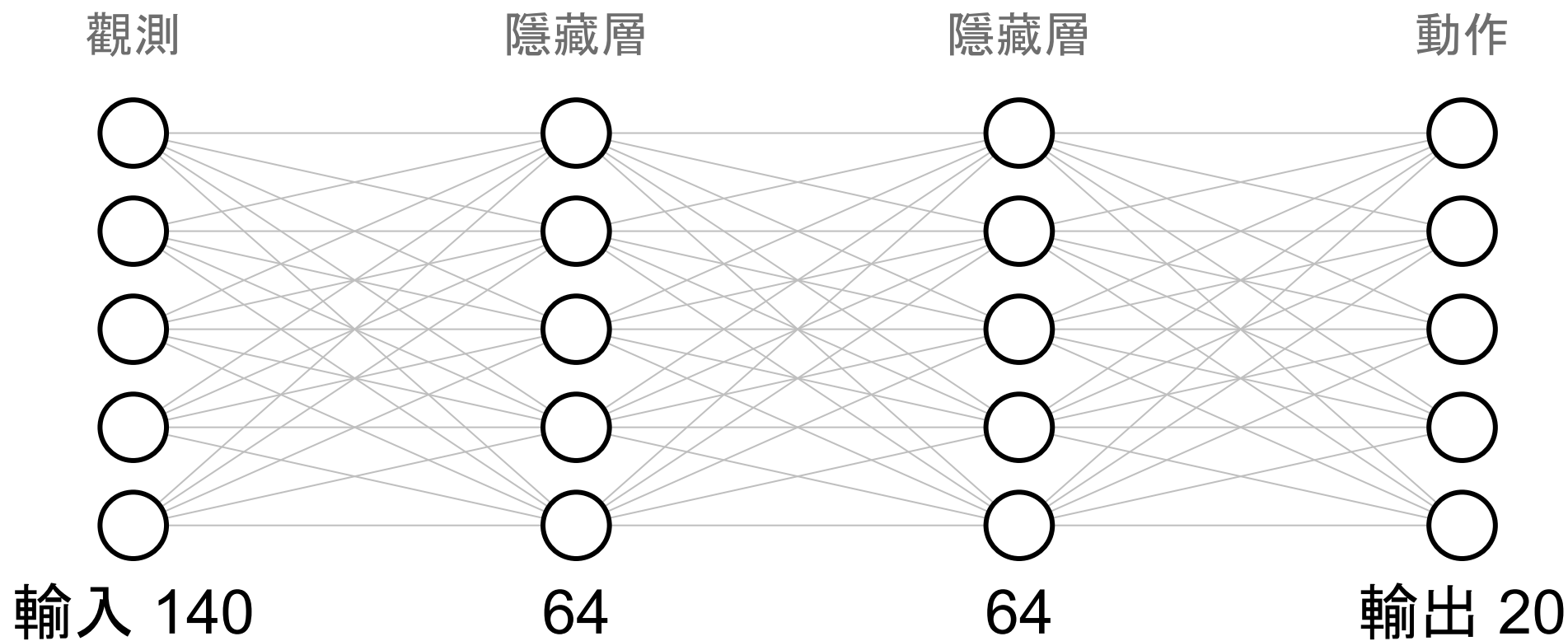
跌倒：扣分並結束



時間到

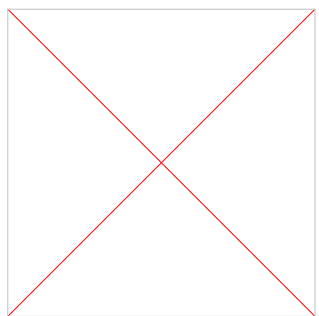
沒跌倒：正常結束

Policy: 一個小小的神經網路



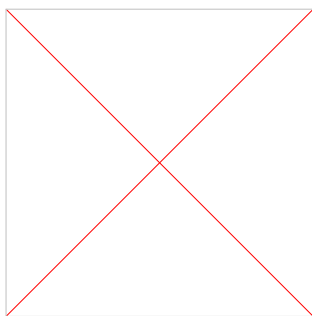
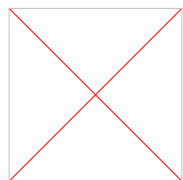
MLP · ELU · 和 BC 的網路一模一樣 → 可以直接注入 PPO

學習怎麼發生？



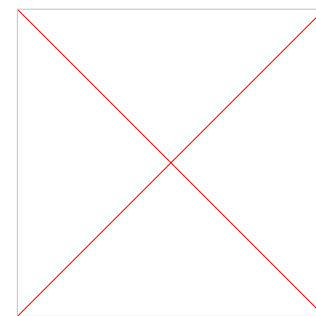
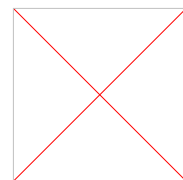
試

用現在的策略
跑一段



評

這個動作比平常
好還是差



改

讓好動作
更常出現



重複幾千次

探索 vs 利用

探索

試沒做過的動作

可能找到更好的走法

太多 → 一直跌倒

利用

用目前最好的動作

穩穩拿分

太多 → 卡在原地

PPO 用動作的隨機程度($\log_std$)控制探索

PPO: 每次只改一點點

±20 %

新舊策略差太多, 就把更新「剪掉」(clip = 0.2)

Proximal Policy Optimization · 機器人強化學習最常用的演算法

學習率：我們量過

$5e-5$

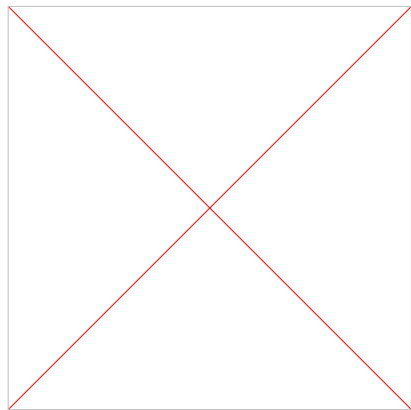
修好一個會跌倒的 BC

$1e-5$

拋光一個已經會走的 BC

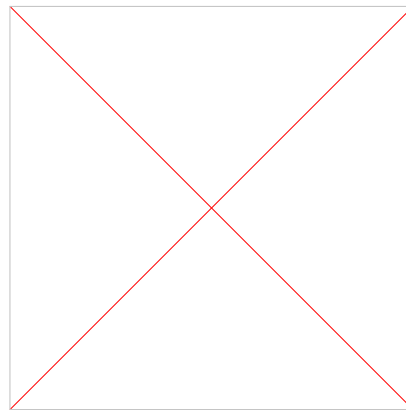
同一個 $5e-5$ ，放在已經會走的模型上，會把它「隨機漫步」到懸崖外

Actor–Critic



Actor 演員

看到觀測
決定做什麼動作



Critic 評審

估計現在的狀態
之後能拿幾分

Critic 讓 Actor 知道：這一步比平常好，還是比平常差

PPO 的重要參數

參數	今天用	白話
n_steps	2048	收集幾步經驗才更新一次
batch_size	256	一次拿多少經驗算梯度
n_epochs	5	同一批經驗重複學幾遍
learning_rate	3e-4	每次改多少
gamma	0.99	多在乎未來的獎勵
clip_range	0.2	每次最多改多少

Track C 實際用: 2048 台 × n_steps 32、batch 8192、學習率 1e-5~5e-5

為什麼要平行？

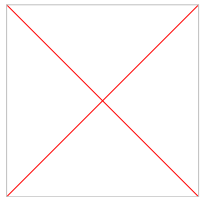
1000 萬步

一次 PPO 訓練要的經驗 (Track C 的設定)

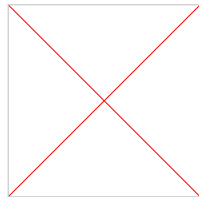
4 × 512 台、每秒 9,300 步 → 約 18 分鐘

只有 1 台 → 好幾個小時

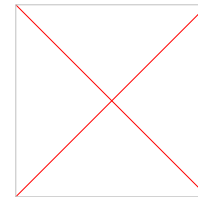
Sim-to-Real: 故意加干擾



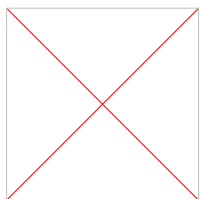
推一把
隨機推撞



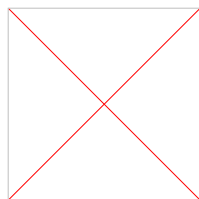
質量
每台都有誤差



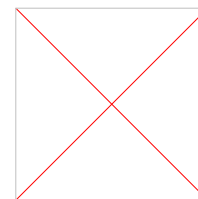
摩擦
地面滑不滑



感測器雜訊
IMU、馬達讀值



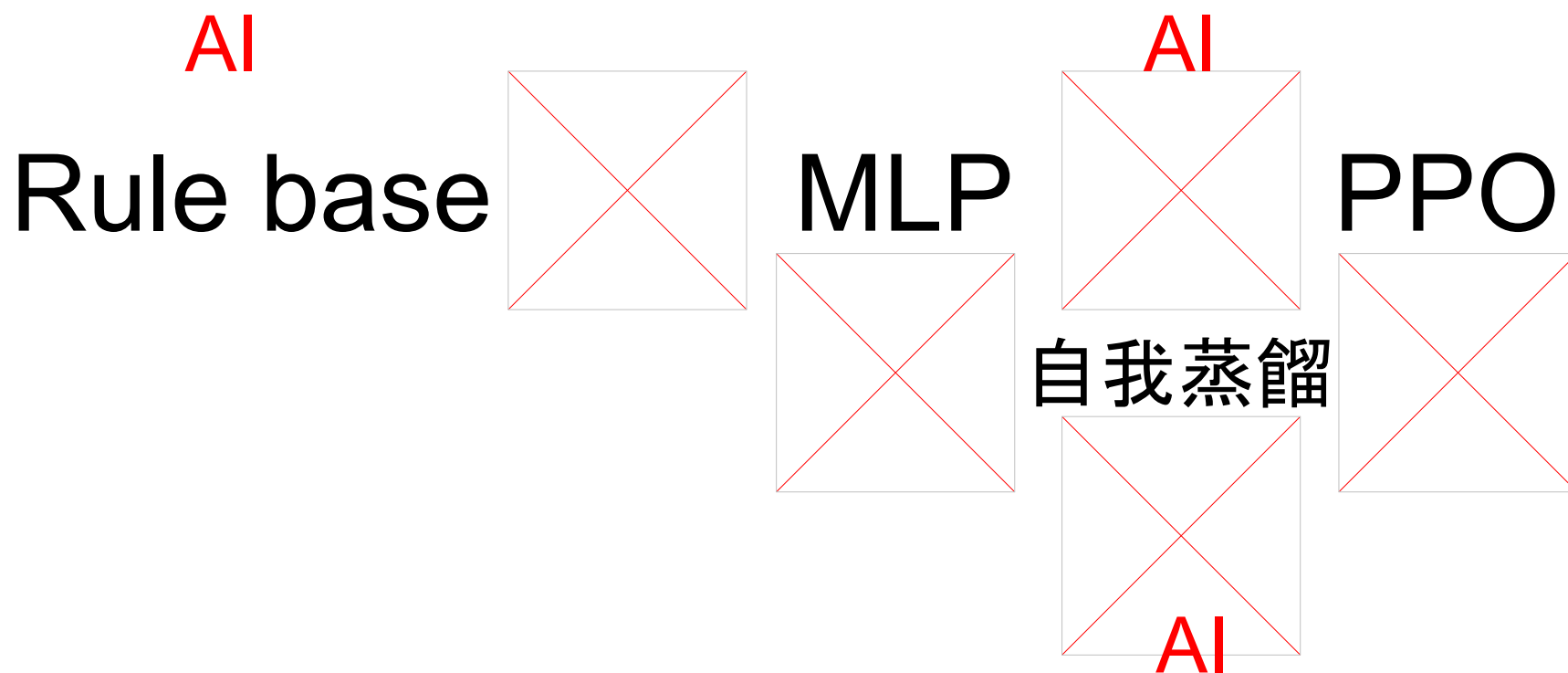
延遲
馬達慢一拍



陀螺儀漂移
航向每分鐘 $\leq 1^\circ$

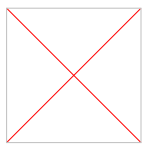
策略在各種「亂一點」的模擬都能走 → 真機只是其中一種 (Domain Randomization)

我們的訓練流程

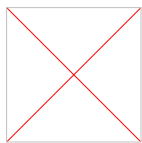


AI = 請 AI 寫的部分

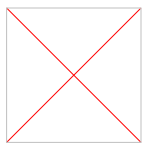
Step 1: Rule base 專家



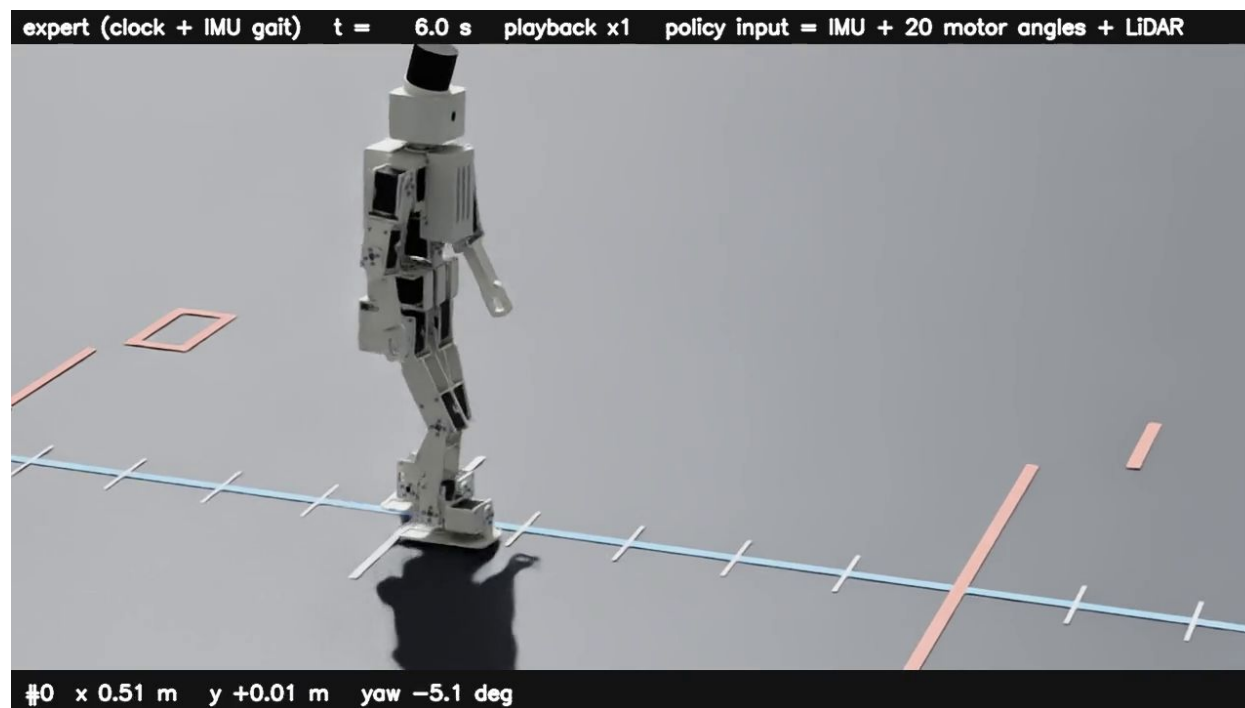
時鐘 + IMU 回授



被推一下就不穩

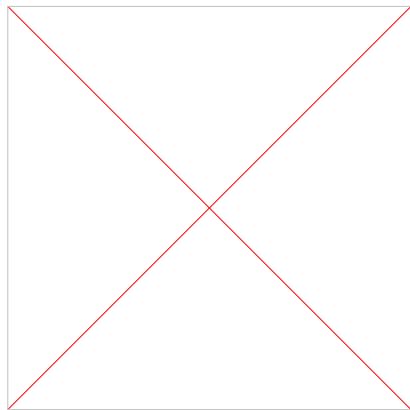


當神經網路的老師



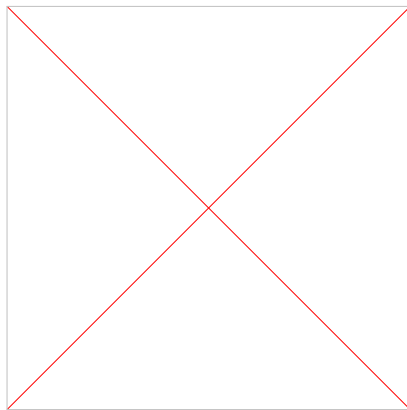
Round 1 的專家(點一下播放)

Step 2: MLP 模仿 (BC)



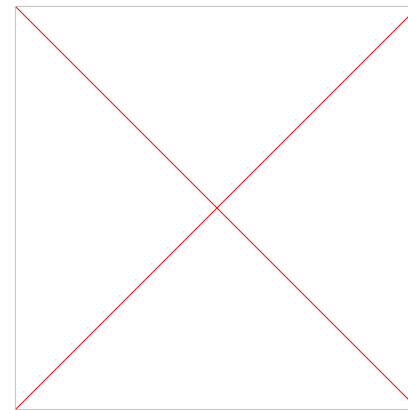
收集

乾淨 + 加雜訊
+ 被推的資料



模仿

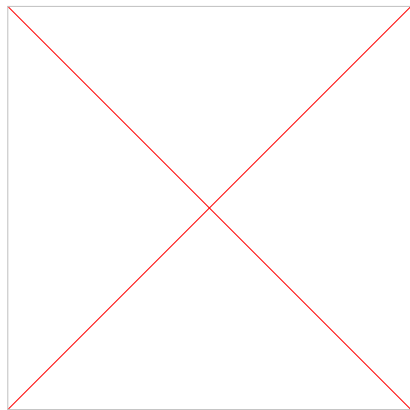
MLP $140 \rightarrow 64 \rightarrow 64 \rightarrow 20$
用 MSE 學一樣的动作



弱點

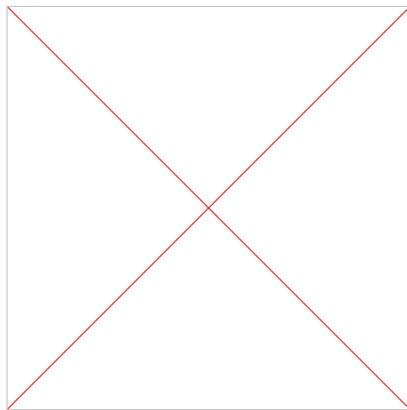
沒看過的狀況
就不會救

Step 3: PPO 微調



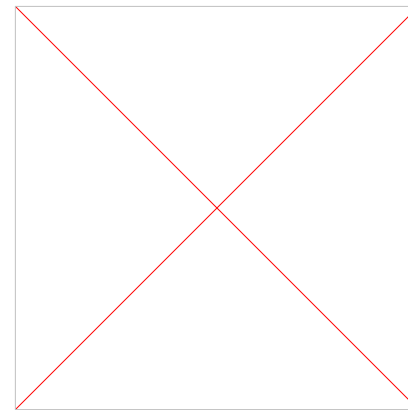
注入

BC 的網路
當 PPO 的起點



練習

2048 台一起
試錯

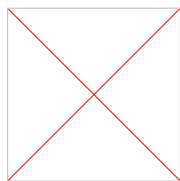


結果

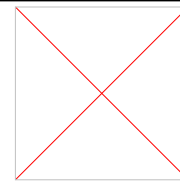
256 / 256 台
走滿 60 秒

Step 4: 自我蒸餾

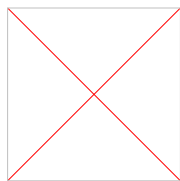
跑 PPO 策略



挑走得好的回合



注入新的 PPO

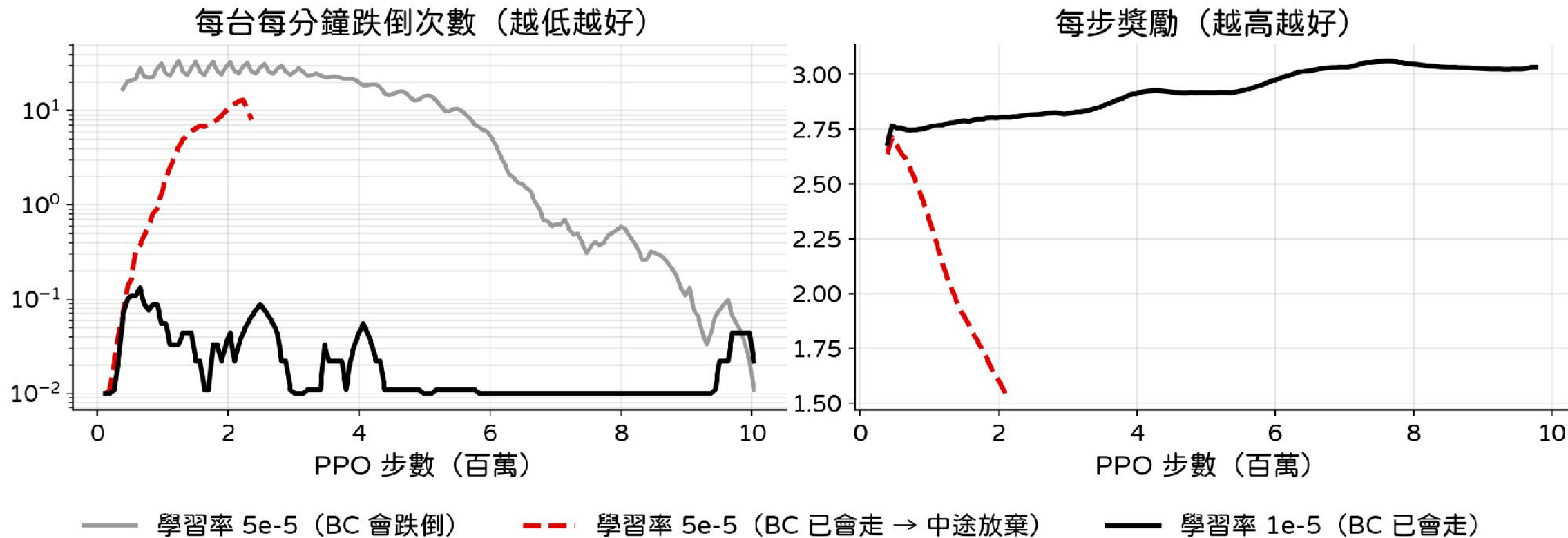


訓練新的 BC

每一圈都重新開始，不是接著練 → 買到的是抗干擾能力

兩倍推撞下能走完的台數: 3 → 28 → 41 → 44 (共 64 台, Round 2)

訓練曲線要看什麼



看「每分鐘跌倒次數」和「每步獎勵」

別看最近 64 回合的存活率:複製體同步起跑, 會在 $0 \leftrightarrow 1$ 之間跳

成果 (Track C)

256/256

60 秒全部沒跌倒

Round 1

79.7 m

600 秒零跌倒

Round 1

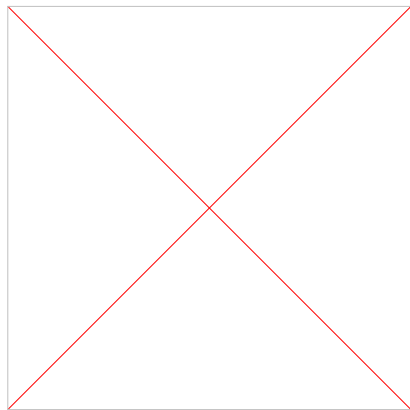
9.2 cm

1600 秒內離線最遠

Round 3

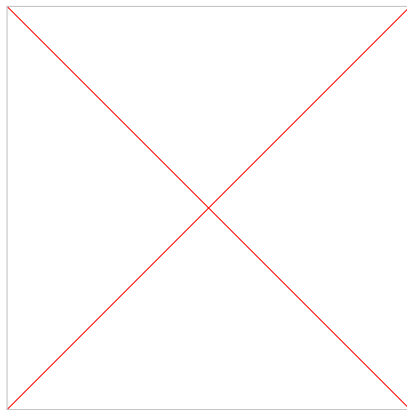
全部只用機器人自己的感測器

Part 4 小結



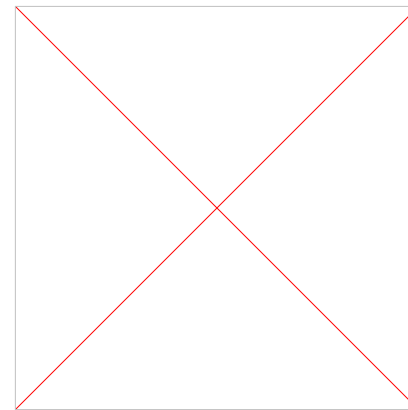
獎勵

決定它學什麼



PPO

每次只改一點點



平行 + 干擾

學得快、學得穩

Part 5

訓練模型

14:00 – 16:00

下午的路線圖

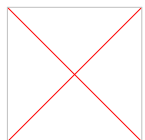


訓練一旦開始就讓它跑，其他事情邊跑邊做

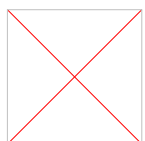
安裝訓練套件

終端機(venv 啟動狀態)

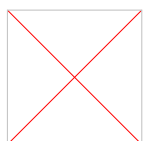
```
pip install stable-baselines3 tensorboard
```



Stable-Baselines3: PPO 演算法



TensorBoard: 看訓練曲線



Gymnasium: 環境的標準介面(跟著一起裝)

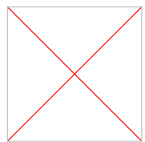
講者的版本: stable-baselines3 2.9.0 · gymnasium 1.3.0 · torch 2.11.0+cu128

檔案結構

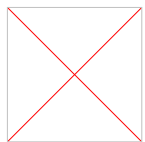
```
workshop/                                路徑不要有中文或空格
├─ Biped_Robot_Master_0925_unity_urdf/   從 Drive 下載的資料夾
│   ├── Biped_Robot_Master_0925.urdf
│   └─ Biped_Robot_Master_0925_description/meshes/
├─ usd/Biped_Robot_Master_0925/          import_urdf.py 產生
├─ robot_usd.txt                         記住 USD 在哪
├─ hello_isaac.py   import_urdf.py   stand.py
└─ walk_env.py     train.py     play.py
```

環境的骨架 (Gymnasium)

```
class WalkEnv(gym.Env):  
    def __init__(self, usd_path, render=False):  
        ...    # 建世界、放機器人、定義 observation / action 空間  
  
    def reset(self, seed=None, options=None):  
        ...    # 回到起點 → return obs, info  
  
    def step(self, action):  
        ...    # 做動作、跑物理 → return obs, reward, terminated, truncated, info
```



terminated: 跌倒 (失敗)



truncated: 時間到 (沒失敗)

walk_env.py (1/6): 設定

```
PHYS_DT, DECIM = 1 / 240, 4          # 物理 240 Hz, 控制 60 Hz
KP, KD = 600.0, 20.0                  # 29 kg 的機器人(Track C 1.2 kg 用 20 / 0.4)
ACT_SCALE = 0.3                       # 動作  $\pm 1 \rightarrow$  目標角  $\pm 0.3$  rad(再夾在關節限位內)
FWD = 1                               # 前方是 +Y(0925 版); Track C 是 0(+X)
V_TARGET, EP_SECONDS = 0.15, 20.0     # 目標速度 (m/s)、每回合秒數
W = dict(alive=1.0, vel=2.0, upright=0.5, rate=0.02, fall=50.0) # 獎勵權重(實作 5-3)
```

```
def rot_inv(q, v):
    """把世界座標的向量 v 轉到機身座標(q = w, x, y, z)"""
    w, u = q[0], q[1:]
    return v * (2 * w * w - 1) - np.cross(u, v) * 2 * w + u * 2 * np.dot(u, v)
```

walk_env.py (2/6): `__init__`

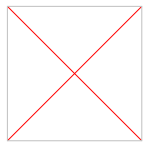
```
class WalkEnv(gym.Env):
    def __init__(self, usd_path, render=False):
        self.render_on = render
        # .....建世界、地板、燈、放機器人、找 articulation(和 stand.py 一樣).....
        self.n = n = self.robot.num_dof
        self.robot.set_gains(kps=np.full((1, n), KP, np.float32),
                             kds=np.full((1, n), KD, np.float32))
        lim = np.asarray(self.robot.get_dof_limits())[0] # (n, 2), 弧度
        self.lo, self.hi = lim[:, 0].astype(np.float32), lim[:, 1].astype(np.float32)
        self.q0 = np.zeros(n, np.float32) # 預設姿勢(改成微蹲會更好學)
        self.p0, self.r0 = self.robot.get_world_poses() # 每回合的起點
        self.max_t = int(EP_SECONDS / (PHYS_DT * DECIM))
        obs_dim = 3 + 3 + n + n + n # 重力、角速度、角度、速度、上一個動作
        self.observation_space = spaces.Box(-np.inf, np.inf, (obs_dim,), np.float32)
        self.action_space = spaces.Box(-1.0, 1.0, (n,), np.float32)
```

walk_env.py (3/6): reset

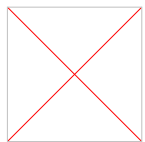
```
def reset(self, seed=None, options=None):
    super().reset(seed=seed)
    self.robot.set_world_poses(self.p0, self.r0)           # 回到起點
    self.robot.set_velocities(np.zeros((1, 6), np.float32)) # 瞬移後速度要歸零
    self.robot.set_joint_positions(self.q0[None])
    self.robot.set_joint_velocities(np.zeros((1, self.n), np.float32))
    self.robot.set_joint_position_targets(self.q0[None])
    self.world.step(render=False)
    self.t, self.last_a = 0, np.zeros(self.n, np.float32)
    return self._obs(), {}
```

walk_env.py (4/6): 觀測

```
def _obs(self):
    q = self.robot.get_world_poses()[1][0]                # 機身姿態(w, x, y, z)
    self.g = rot_inv(q, np.array([0.0, 0.0, -1.0]))        # 重力方向(IMU)
    gyro = rot_inv(q, self.robot.get_angular_velocities()[0]) # 角速度(陀螺儀)
    qj = self.robot.get_joint_positions()[0] - self.q0     # 馬達角度
    qd = self.robot.get_joint_velocities()[0]              # 馬達轉速
    obs = [self.g, 0.25 * gyro, qj, 0.05 * qd, self.last_a]
    return np.concatenate(obs).astype(np.float32)
```



重力方向、角速度 → IMU



角度、轉速 → 伺服馬達回報

walk_env.py (5/6): step

```
def step(self, action):
    a = np.clip(action, -1.0, 1.0).astype(np.float32)
    target = np.clip(self.q0 + ACT_SCALE * a, self.lo, self.hi)[None]
    for _ in range(DECIM):                                # 同一個目標, 物理跑 4 步
        self.robot.set_joint_position_targets(target)
        self.world.step(render=False)
    if self.render_on:
        self.world.render()
    self.t += 1
    rate = float(np.sum((a - self.last_a) ** 2))
    self.last_a = a
    obs = self._obs()
```

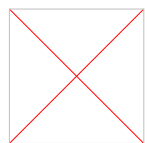
walk_env.py (6/6): 獎勵與結束

```
v = float(self.robot.get_linear_velocities()[0, FWD]) # 前進速度(真值, 只算獎勵)
upright = float(-self.g[2]) # 1 = 完全直立
fallen = upright < 0.707 # 傾斜超過 45° 就算跌倒
reward = (W["alive"]
          + W["vel"] * np.exp(-((v - V_TARGET) / 0.1) ** 2)
          + W["upright"] * upright
          - W["rate"] * rate
          - (W["fall"] if fallen else 0.0))
truncated = self.t >= self.max_t
return obs, float(reward), bool(fallen), bool(truncated), {}
```

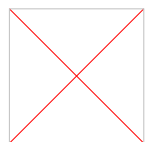
train.py (1/2): 先加速

```
from isaacsim import SimulationApp
app = SimulationApp({"headless": True})           # 訓練不開畫面, 比較快

import carb                                       # 不把結果寫回 USD:快 2.5 倍
carb.settings.get_settings().set("/physics/updateToUsd", False)
carb.settings.get_settings().set("/physics/updateVelocitiesToUsd", False)
from stable_baselines3 import PPO
from stable_baselines3.common.callbacks import CheckpointCallback
from walk_env import WalkEnv                    # 一定要在 SimulationApp 之後
```



不把物理結果寫回 USD:實測每秒 75 步 → 187 步



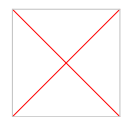
物理結果完全一樣;play.py 要看畫面, 所以不關

train.py (2/2): 開始訓練

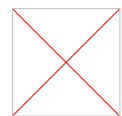
```
usr = sys.argv[1] if len(sys.argv) > 1 else open("robot_usr.txt").read().strip()
env = WalkEnv(usr)
model = PPO("MlpPolicy", env,
            n_steps=2048, batch_size=256, n_epochs=5,
            learning_rate=3e-4, gamma=0.99, gae_lambda=0.95, clip_range=0.2,
            policy_kwargs=dict(net_arch=[64, 64], log_std_init=-1.0),
            tensorboard_log="runs", device="cpu", verbose=1)
model.learn(total_timesteps=2_000_000,
            callback=CheckpointCallback(save_freq=50_000, save_path="checkpoints",
                                       name_prefix="walk"))
model.save("walk_ppo")
os._exit(0)
```

組出環境、開始訓練

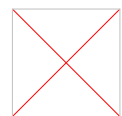
實作 5-1



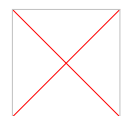
`walk_env.py`、`train.py` 放進資料夾



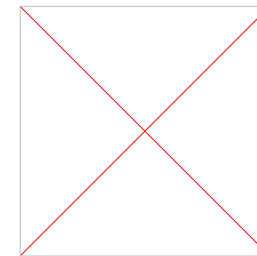
`python train.py`



終端機開始印出 `ep_rew_mean`



`checkpoints/` 每 5 萬步多一個檔



30
min

訓練會一直跑，不要關掉；接下來開 TensorBoard

TensorBoard: 看訓練曲線

另開一個終端機(同一個 venv)

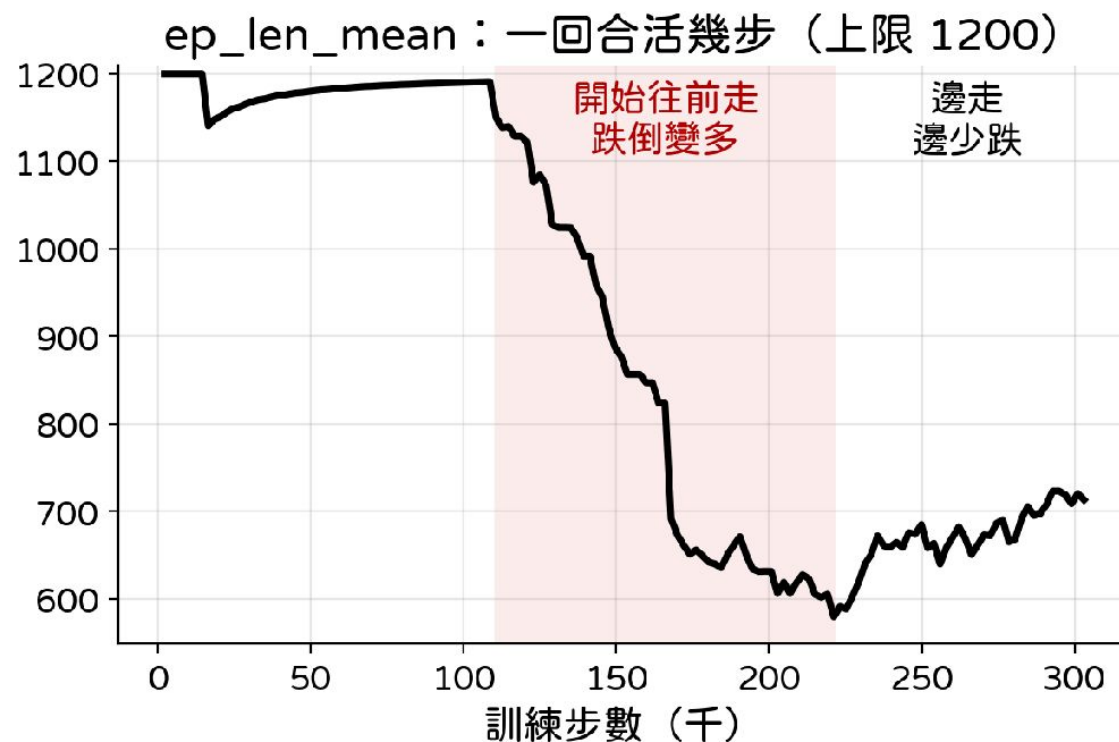
```
tensorboard --logdir runs
```

瀏覽器打開

localhost:6006

指標	意思	怎麼看
rollout/ep_rew_mean	一回合平均拿幾分	長期往上
rollout/ep_len_mean	一回合平均活多久	跌倒多就變短
train/approx_kl	新舊策略差多少	不要突然變大

實測：照做訓練 30 分鐘



30 萬步後播放：20 秒平均往前 1.3 m, 4 次都沒跌倒

play.py: 看它學了什麼

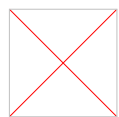
```
import sys
from isaacsim import SimulationApp
app = SimulationApp({"headless": False})
from stable_baselines3 import PPO
from walk_env import WalkEnv
model = PPO.load(sys.argv[1] if len(sys.argv) > 1 else "walk_ppo", device="cpu")
env = WalkEnv(open("robot_usd.txt").read().strip(), render=True)
obs, _ = env.reset()
while app.is_running():
    action, _ = model.predict(obs, deterministic=False) # 跟訓練時一樣帶雜訊
    obs, reward, terminated, truncated, _ = env.step(action)
    if terminated or truncated:
```

訓練中也能播: `python play.py checkpoints/walk_200000_steps.zip`

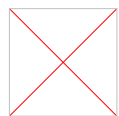
改成 `deterministic=True` 再播一次: 它只會站著——為什麼?

看曲線、播放

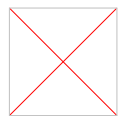
實作 5-2



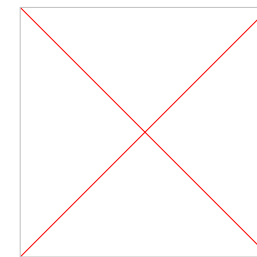
TensorBoard 看到曲線在動



用最新的 checkpoint 播放

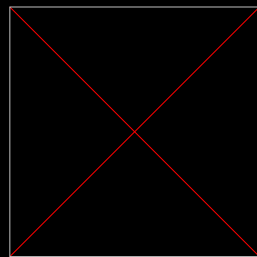


比較 5 萬步和最新的：哪裡不一樣？



15
min

曲線往下掉不一定是變差：播放看看它在做什麼



休息一下

讓模型繼續跑，人休息 10 分鐘

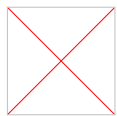
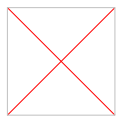
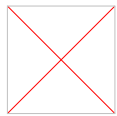
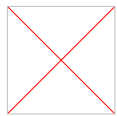
改一個獎勵，看行為怎麼變

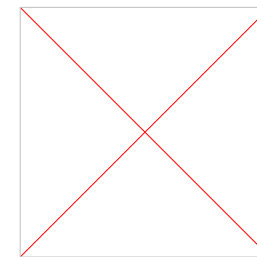
改哪裡 (walk_env.py)	你可能會看到
<code>W["vel"]</code> 2.0 $\rightarrow$ 6.0	更衝，也更早跌倒
<code>W["upright"]</code> 0.5 $\rightarrow$ 0	身體歪歪地走
<code>W["rate"]</code> 0.02 $\rightarrow$ 0.5	動作變慢、變平滑
<code>W["alive"]</code> 1.0 $\rightarrow$ -1.0	可能學會「趕快倒下」
<code>V_TARGET</code> 0.15 $\rightarrow$ 0.3	想走快，但更容易跌

先寫下你的預測，再訓練 $\rightarrow$ 猜錯的地方最有收穫

改獎勵、重新訓練

實作 5-3

-  選一個改動，先寫下預測
-  改 `walk_env.py` 的 `W`
-  重新 `python train.py`
-  TensorBoard 比較兩次的曲線



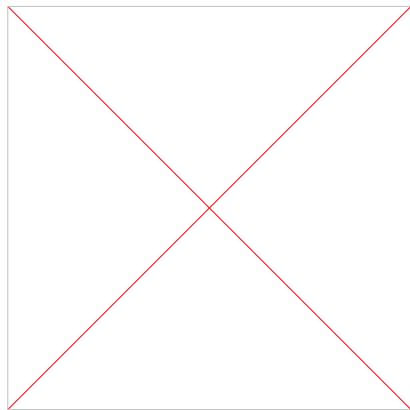
25
min

一次只改一個地方，不然不知道是誰造成的

常見失敗與對策

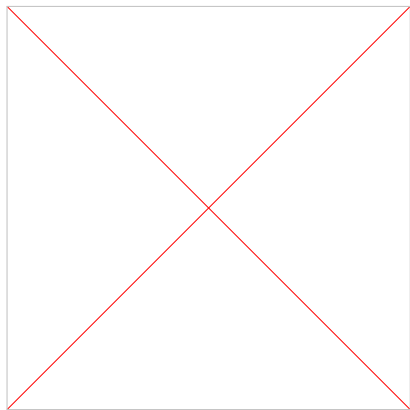
現象	可能原因	怎麼辦
一開始就彈飛	地板有彈性、生成太低	restitution 0、離地 2 mm
往旁邊或往後走	前進方向設錯	量一下, 改 FWD
視窗全黑	場景裡沒有燈	加 DomeLight
原地踏步不前進	站著最安全	速度獎勵更「尖」
學會了又忘掉	學習率太大	降低學習率
曲線一直很平	探索太少或太多	調 log_std_init

讓訓練變快



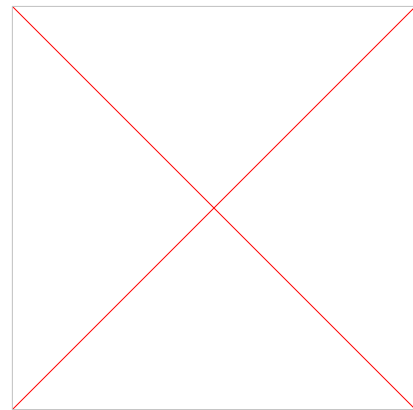
headless

不開畫面



不寫回 USD

實測快 2.5 倍



一個場景很多台

一個 view 控制全部

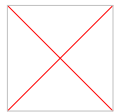
進階：一個場景放很多台

改寫自 Track C 的 sim_c.py

```
for i in range(n):
    prim = add_reference_to_stage(usd_path=USD, prim_path="/World/env_%02d" % i)
    UsdGeom.XformCommonAPI(prim).SetTranslate(Gf.Vec3d(x[i], y[i], z0)) # 建立時就放好
robots = Articulation(prim_paths_expr="/World/env_*/Geometry/base_link", name="robots")
world.scene.add(robots)
world.reset()
robots.set_joint_position_targets(targets) # shape = (n, 20):一次控制全部
```



名稱要補零 env_02:regex 照字母排序, env_10 會排在 env_2 前面



光達射程要小於機器人間距:場景查詢會看到隔壁的機器人

GPU 使用率 $\neq$ 速度

配置	GPU 使用率	速度
4 行程 $\times$ 64 台	84–88 %	$\approx$ 3,800 步/秒
4 行程 $\times$ 512 台	39–41 %	$\approx$ 9,300 步/秒

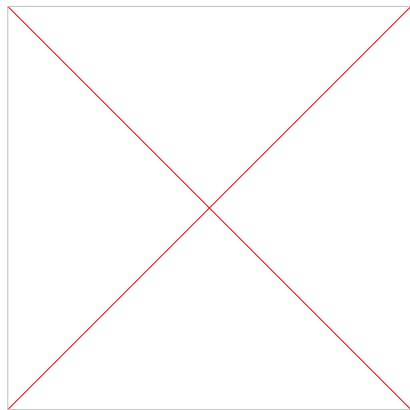
瓶頸是 PhysX 每台每步的 CPU 成本 $\rightarrow$ 看「步/秒」, 別看 GPU %

RTX 5070 Ti · Track C 第一輪實測

我不寫 code

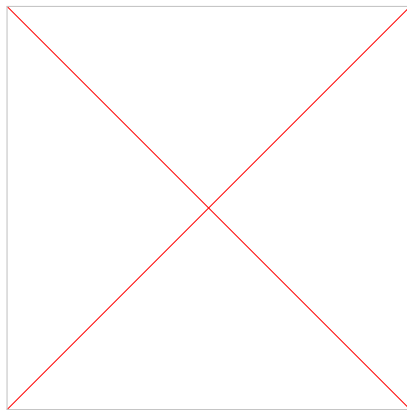
讓 AI 幫你寫，但你要知道它會錯在哪

跟 AI 合作的三個原則



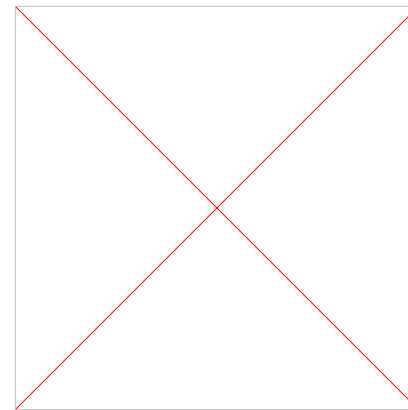
限縮範圍

講清楚版本、檔案、目標



不要中斷

讓它一次做完、自己驗證



記取教訓

把踩過的坑寫進 prompt

好的 prompt 長這樣

不好的

幫我寫一個
讓機器人走路
的程式。

好的

@workshop 裡有 0925 版的 URDF
和 walk_env.py。用 Isaac Sim 6.0.1
(Python 3.12) 的 API, 把前進速度的
獎勵改成.....。改完自己跑 1 萬步,
確認沒有錯誤、曲線有出來再告訴我。

給 AI 的防呆清單

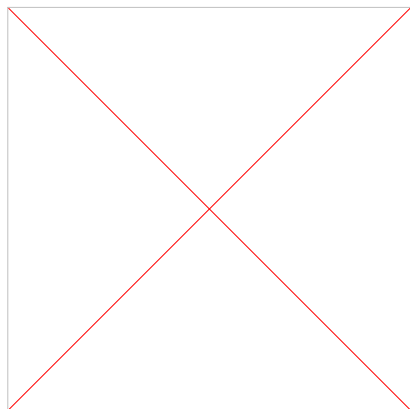
直接貼進 prompt

- Isaac Sim 6.0.1:URDF 匯入用 URDFImporter / URDFImporterConfig (5.x 的 `_urdf` 已移除, 網路上的舊寫法不能用)
- SimulationApp 必須在所有 isaacsim / omni 的 import 之前建立
- 地板 restitution 設 0(預設 0.8 會彈);自己建的地板要加燈, 不然畫面全黑
- 前進方向要量:Biped_Robot_Master 是 +Y、Track C 是 +X
- base_link 原點可能在腳底:跌倒用傾斜角判斷
- 瞬移後把速度歸零;print 加 flush=True;結束用 `os._exit(0)`
- 觀測只能用真機量得到的(IMU、馬達角度);真值只拿來算獎勵

今天踩到的新坑, 也加進你自己的清單

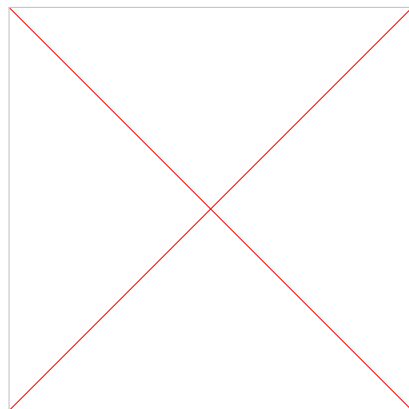
挑戰題

實作 5-4



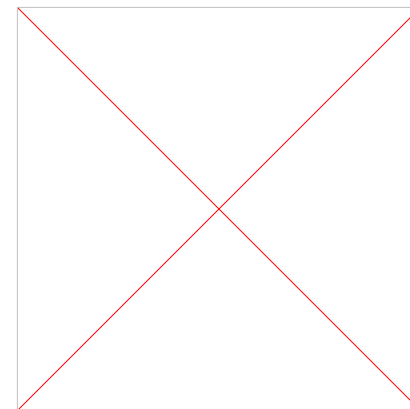
★ 被推也不倒

每 2 秒推一把
set_velocities



★★ 走 1 公尺

調獎勵
訓練久一點

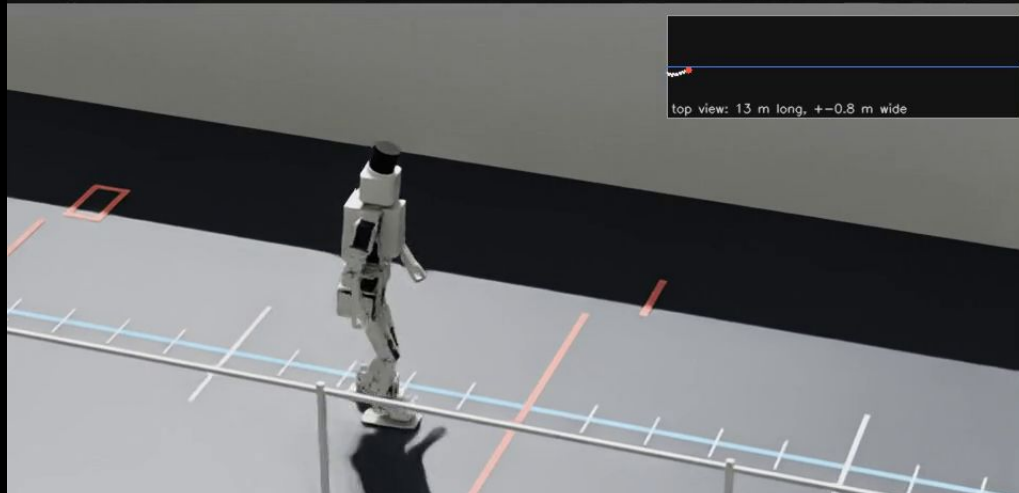


★★★ 走直線

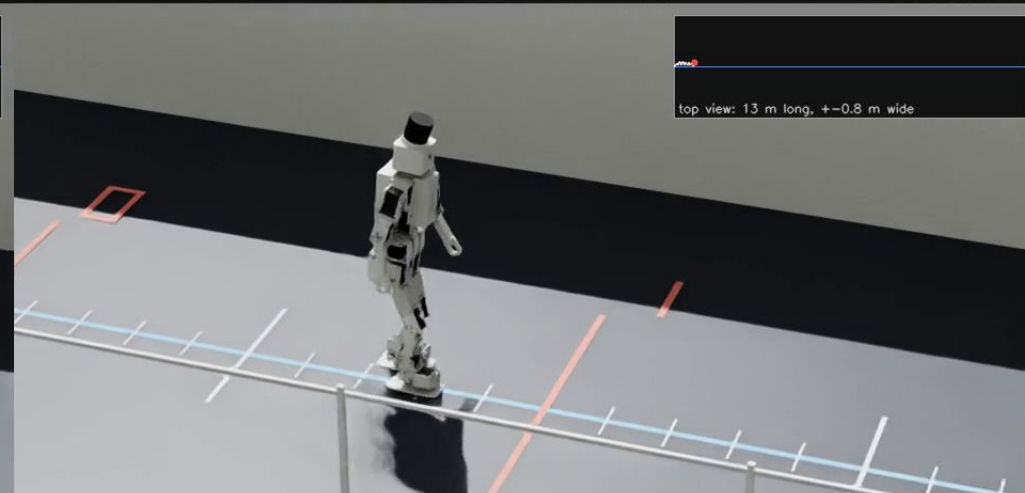
1 公尺內
左右偏離 < 5 cm

為什麼沒有「單腳站」？這台沒有踝關節 roll，我們量過：做不到

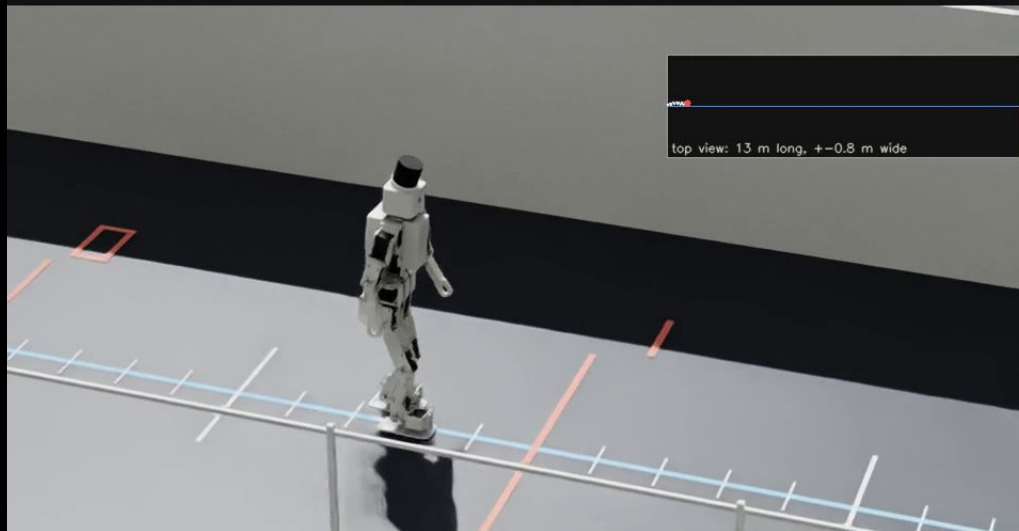
rl3 (PPO actor) $t = 6.0 \text{ s} \times 1$ input: IMU, motor angles, camera line + LiDAR wall detection [kicked $0.05 \text{ m/s} / 5 \text{ s}$]



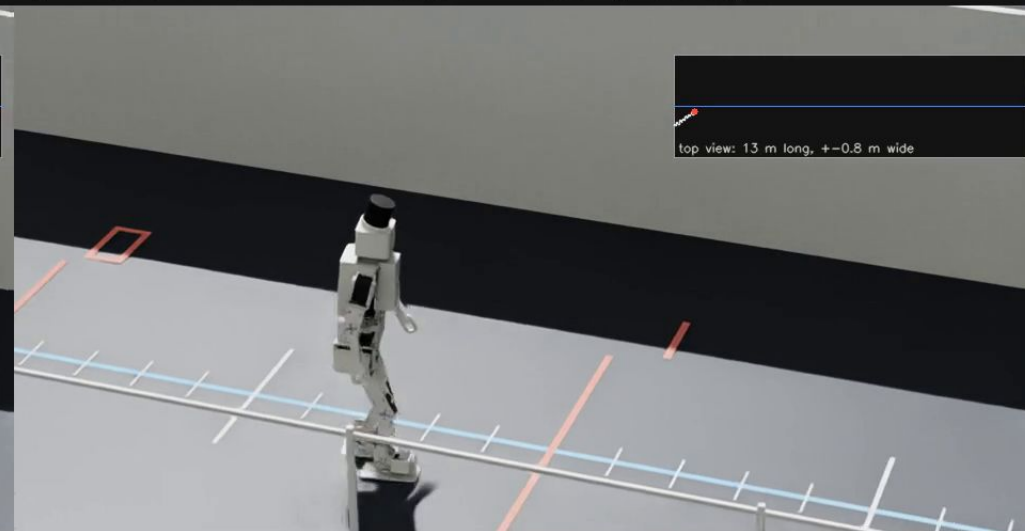
#0 x 0.77 m off the line -0.051 m heading $+5.9 \text{ deg}$



#1 x 0.76 m off the line $+0.051 \text{ m}$ heading -2.7 deg



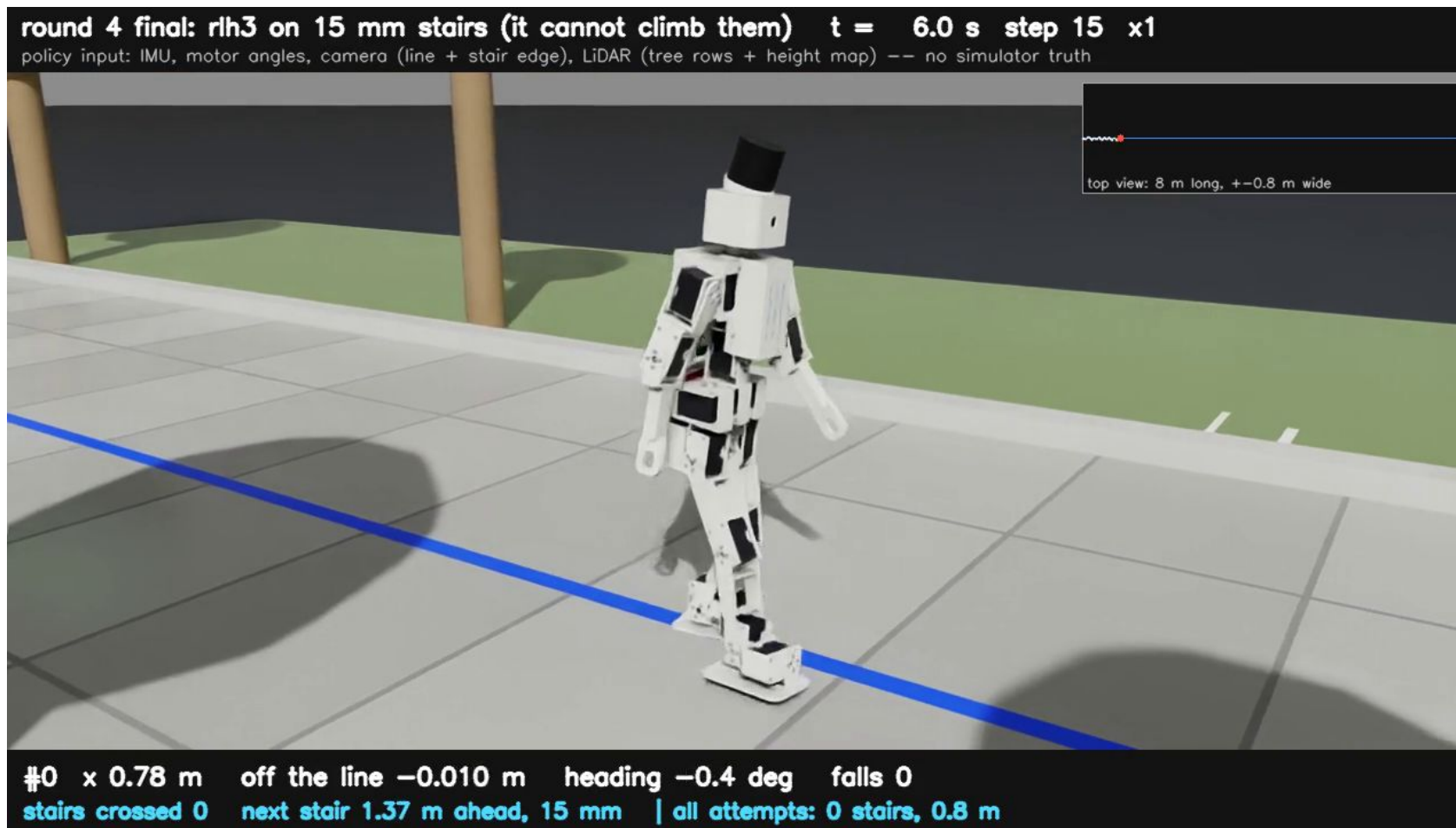
#2 x 0.76 m off the line $+0.041 \text{ m}$ heading -2.3 deg



#3 x 0.75 m off the line -0.097 m heading $+10.7 \text{ deg}$

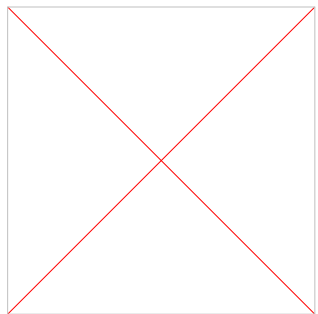
Round 2 的 PPO: 每 5 秒被踢一下 (0.05 m/s), 照樣走回線上

下一個挑戰：樓梯



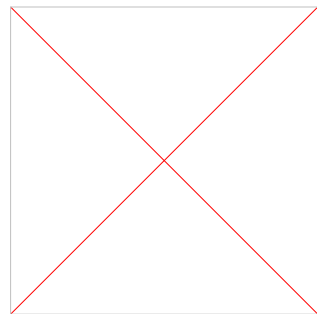
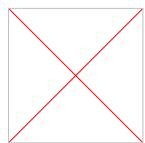
8 mm 的台階 87 % 過得去, 15 mm 還是 0 % —— 研究還在進行

從模擬到真機



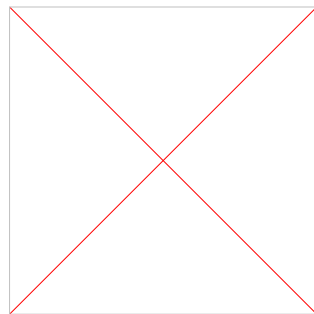
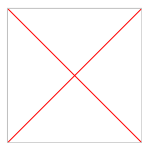
模擬訓練

Isaac Sim + PPO



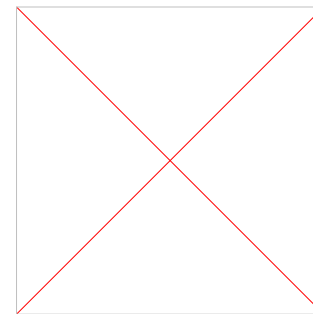
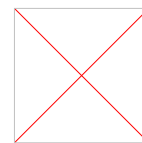
加干擾

雜訊 · 延遲 · 推撞



只看感測器

IMU · 馬達
光達 · 相機



真機

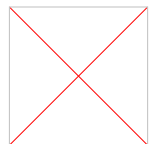
Raspberry Pi 5
20 顆 XC330

成果分享

你的機器人學會了什麼？

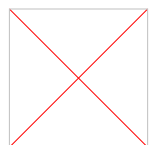
曲線 · 影片 · 你改了什麼

今天學到的



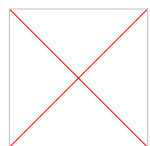
認識 Isaac

模擬：便宜、快、可以平行



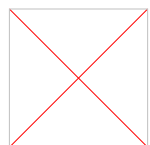
安裝

Python 3.12 + pip install isaacsim



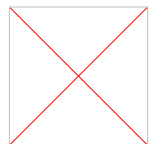
URDF

URDF → USD, 匯入後一定要驗證



強化學習

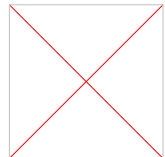
獎勵決定它學什麼



訓練

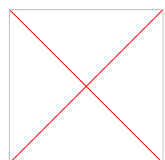
環境 → 訓練 → 看曲線 → 改獎勵

學習資源



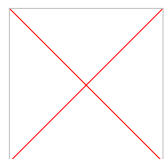
Isaac Sim 文件

docs.isaacsim.omniverse.nvidia.com



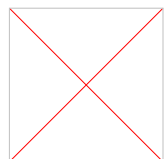
Isaac Lab

isaac-sim.github.io/IsaacLab



Stable-Baselines3

stable-baselines3.readthedocs.io

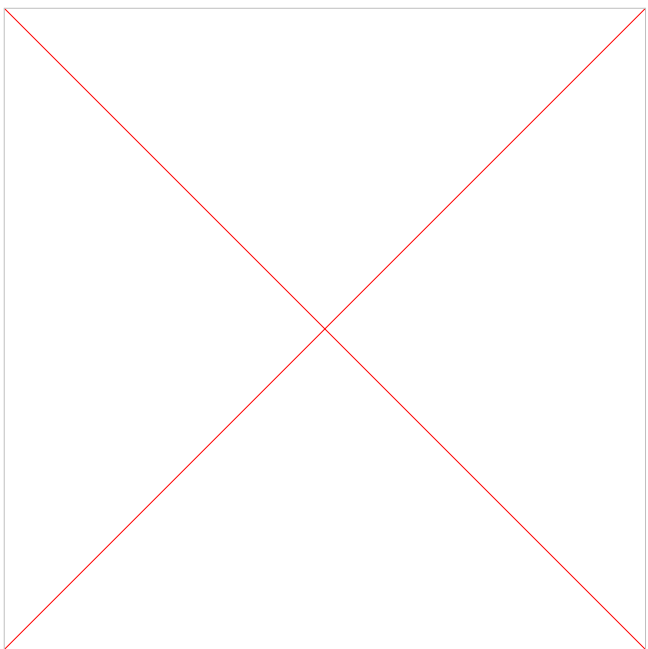


Gymnasium

gymnasium.farama.org

Q & A

期待大家的機器人 都能跑啊跑



講者：王羿程